DTU ELECTRO
Department of Electrical and Photonics Engineering

Vision-Guided Robotic System for Real-Time Object Tracking and Handling

Master Thesis

Karol Chudzicki

2025

Vision-Guided Robotic System for Real-Time Object Tracking and Handling

By
(s232675) Karol Chudzicki

Advisor(s)
Søren Hansen, Associate Professor at the Electrical Engineering and Photonics Department of DTU

Education Programme	Master Thesis Electrical Engineering
Period	28 January 2025 to 15 August 2025
ECTS	30

Copyright:	Karol Chudzicki 2025 Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.

Cover photo:	Christian Andersen, 2018
Published by:	DTU, Department of Electrical and Photonics Engineering, Elektrovej 326, 2800 Kgs. Lyngby Denmark https://www.electro.dtu.dk
Remark	This report is submitted as partial fulfillment of the requirements for graduation in the above education at the Technical University of Denmark.

Abstract

Although the thesis is titled Robotic Sorting and Assembly, the work has focused heavily on developing the supporting subsystems — particularly computer vision and motion control — which are critical enablers for robust sorting and assembly operations.

Contents

Abstract ii

1 Introduction **1**
1.1 Motivation 1
1.2 Objectives 2
1.3 Limitations 2

2 Problem analysis **5**
2.1 State of the art 5
2.2 Decisions 7
2.3 Summary 7

3 Product design: power supply **9**
3.1 Product design choice 9
3.2 Design 10

4 Robotic system architecture **11**
4.1 Robotic arm 12
4.2 Conveyor belt 13
4.3 Gripper 13

5 Vision: camera and algorithms **15**
5.1 Related work 15
5.2 Camera Calibration 15
5.3 Hand eye calibration 19
5.4 Vision system methodology 22
5.5 Image processing pipeline 22
5.6 Part classification and angle estimation 25
5.7 Center pose tracking 26
5.8 Graphical user interface (GUI) 31

6 Control systems: robot motion and trajectory planning **33**
6.1 Methodology 33
6.2 Standby 35
6.3 Pursuit 40
6.4 Descend 46

7 System test **49**
7.1 Performance tests 49

8 Conclusion **51**

Bibliography **53**

List of Abbreviations

YOLO	You Only Look Once
IBVS	Image-based Visual Servoing
PBVS	Position-based Visual Servoing
UR5	Universal Robots robotic arm (5 kg payload)
Robotiq 2F-85	Robotiq two-finger gripper (85 mm maximum opening)
TCP/IP	Transmission Control Protocol / Internet Protocol
RTU	Remote Terminal Unit (Modbus communication standard)
RS-485	Recommended Standard 485 (serial communication standard)
AC	Alternating Current
DC	Direct Current
OpenCV	Open Source Computer Vision Library
HSV	Hue, Saturation and Value colour space
CLAHE	Contrast Limited Adaptive Histogram Equalization
LAB	Lightness, A-Channel (green-red), B-Channel (blue-yellow) Colour Space
JSON	JavaScript Object Notation
GUI	Graphical User Interface
ROI	Region of Interest
FOV	Field of View
PnP	Perspective-n-Point
X, Y, Z	Cartesian Coordinate Axes
R_x, R_y, R_z	Rotations about the X, Y, Z axes
RPM	Rotations Per Minute
RPS	Rotations Per Second
freq / f	Frequency
v	Velocity
r	Radius
STD	Standard Deviation
RSD	Relative Standard Deviation
v_x	Velocity along the X axis of the robotic arm
v_y	Velocity along the Y axis of the robotic arm
v_z	Velocity along the Z axis of the robotic arm
max	Function that returns the bigger value of the two.

1

Introduction

1.1 Motivation

As modern industrial automation has developed, more and more tasks have been taken over by robotic arms. Robotic arms have taken tasks that were once performed by people, especially repetitive, exhausting, or dangerous tasks. This has led not only to more efficient and faster production rates, with a reduction in human errors, but also expanded what can be achieved in manufacturing. robotic arms have made it possible to manufacture more advanced products with greater efficiency and intelligence. Key jobs performed by robotic arms involve object tracking, handling and manipulation. Today industrial robotic arms are ubiquitous in sectors such as electronic assembly or car manufacturing.[1]

A big breakthrough in industrial automation was the implementation of computer vision. Humans depend on their sight to locate objects and navigate in the surroundings, and in the same way, robotic arms can benefit from integration of visual systems to detect and localize objects in their workspace. Adding vision to robotic arms enabled them to adapt to their operating area, and not only perform preprogrammed movements, but also adjust them based on the real-time input from the environment. The same way as humans react when something falls to the ground, robotic arms can be programmed to notice it and react accordingly using their visual input. In a more complex scenario, when an object is moving, humans can follow and catch it using their sight. Similarly, robotic arms can rely on visual system or other sensors to do the same. [2]

This advancement is part of a wider transformation known as Industry 4.0. It's often called the 4th industrial revolution and revolves around introducing smart factories where all the machines and sensors communicate with each other to optimise the process of the tasks they do. Vision in this context plays a crucial role by adding to the system the ability to make decisions based on visual input. By giving robotic arms the ability to 'see' and make decisions based on the data from their surroundings, manufacturers have gained enormous flexibility and can automate a wider range of tasks. [3]

This project is inspired by the development of industrial automation and computer vision. It's meant to explore what is possible within a professional setup using an industrial robot, conveyor belt and a gripper, while using a simple web-camera and a personal PC as

a main processing system. In a typical industrial setting complex vision setups are employed, such as 3D stereo cameras or multi-camera configurations, to deliver accurate object position [4]. This project provides insights into difficulties and bottlenecks of using a basic vision system for robotic manipulation tasks. It's meant to explore the feasibility of designing a practical robotic system and offers findings on potential upgrades and improvement that could be made with more resources, such as higher-quality camera, computer with more computational power or additional sensors.

The goal of this project was to simulate an industrial assembly environment by developing a robotic system capable of grasping parts scattered on a conveyor belt using a vision system and placing them in designated locations. The code for this project was designed with a focus on a modular structure. Each key functionality in this project was divided into modules, allowing for simple modification or even replacement if needed, without affecting other parts of the code. This approach also presents best practices in the design of the automation software.

Before the work started on the project, several objectives were established to clearly define the work that had to be done.

1.2 Objectives

The main objectives of this thesis, listed by the priority are:

1. **Part** - Choosing, designing and 3D printing a product.
2. **Supporting structures** - Constructing supporting structures such as storage unit and mount for the camera.
3. **Image processing** - Create an image processing algorithm for detecting and categorising the components as well as pinpointing the key points for guiding the robot.
4. **Robot control** - Create a code for controlling the robot movements based on the information and data from the image processing algorithm. It should be robust and precise, ensuring that the parts are reliably picked up from the conveyor belt and moved to a designated slot in the storage unit.
5. **System integration** - Create a code that will integrate parts of the systems together: robot, conveyor belt, camera and gripper.
6. **Testing** - Test the complete system and improve its performance if possible.

1.3 Limitations

In this project some limitations were set. These were introduced either because of physical constraints or as a way to simplify the project to avoid unnecessary complexity. By setting those restrictions the work in this project remained focused on the most crucial objectives and could be completed within available time frame. The limitations are as follows:

- **Simplified product design** - The product designed and handled by the robot in this project is only a representation of an actual component. It is intended to simulate the dimensions and size of an actual product that could be found in a real-world setting. Its width in the widest section had to also be smaller than the width of the conveyor belt to prevent the product from extending beyond its boundaries. It's important to note that the product is only a simplification and/or representation and was inspired by the shape of a real-world component. The aim of this project was to demonstrate that the system can handle an object of similar size to real-world item, rather than recreate the complexity of actual product.

- **Limited component number** - The designed product also has a limited number of components. This focused the project more on the robot movement control rather than handling many intricate parts as commonly done in real-world settings.
- **Product colours** - The colour of the product has to stand out from the background in order to make it easier for the vision system to detect it. Therefore it may not resemble colours used in real-world scenarios.
- **Grasping area** - The area where the product could be picked up by the robot from the conveyor belt was also limited, due to robot's reach and conveyor belt length. The system had to be designed according to this constraint. The graphic below presents the dimensions of this area. The area limited area in the figure 1.1 envelops straight part of the conveyor belt after the first bend and before the second bend. It was set to be as long as possible. The limiting factors were the vision system, and the robotic arm reach.

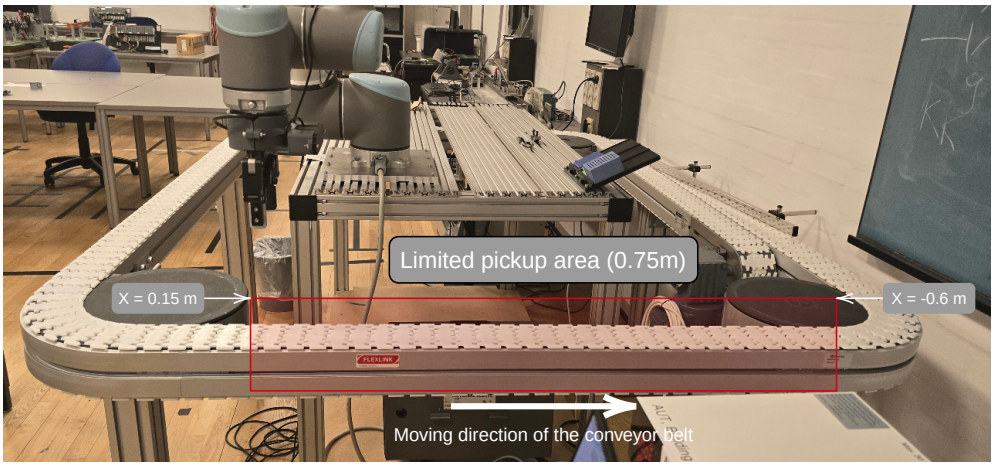


Figure 1.1: Limited pickup area

- **Single camera** - The project relies on a single camera to detect and localize objects. A 3D vision system could provide the system with more accurate and precise data, however it would add additional complexity. For the scope of this project one camera was deemed sufficient, since 3D vision was not within the focus of this thesis.
- **Gripper geometry** - The gripper used for this project had a limited maximum object width that it could grasp and handle. Therefore, the designed parts had to be created according with this constraint and had to be smaller than the gripping width. The maximum gripper range is illustrated in the figure 1.2.
- **Processing Power Limitations** - The computer used in this project had limited processing resources which particularly affected the performance of vision algorithms. The vision algorithms had to be optimised according to these restrictions to prevent significant latency issues during the operation of the system.

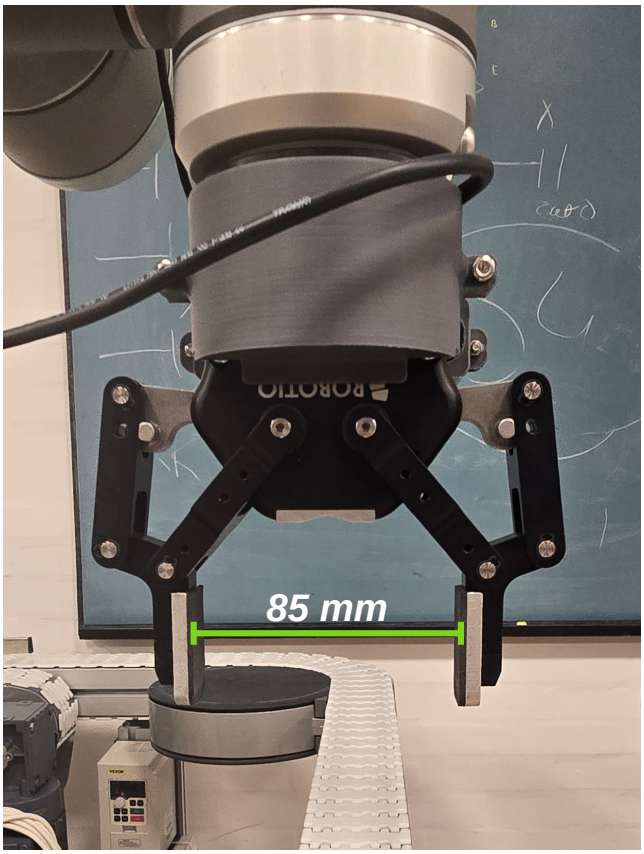


Figure 1.2: Maximum gripper range

2

Problem analysis

2.1 State of the art

Vision-guided robotic systems are ubiquitous in many manufacturing settings, where efficiency and reliability are crucial. Unlike traditional robotic systems relying on fixed taught positions, the integration of a vision system provides additional flexibility. Fixed positions can be adjusted, and the robot can even use visual guidance to move to completely new positions based on the visual localization.

2.1.1 Example Applications of Robotics and Vision in industry

Automatic systems for assembly are widely used to manufacture everyday products. For example, in the paper [5] an author presents a solution for assembling mobile phones using an industrial robot with 6 degrees of freedom. This system uses a stationary camera mounted above the conveyor belt to detect screen and shell components for cell phones and estimate their position for the robotic arm to grab them. The system demonstrated that it is possible to achieve a 100% pickup rate and a 98% assembly success rate for mobile phone components in real industrial environment.

Robotic assembly systems can also be applied to large industrial components as demonstrated in the paper [6]. In this system, the robot is used for a novel idea of assembling wind turbine blades using sensor feedback. The panels of the blades are picked up by the robot using a vision system mounted to the arm and then precisely guided to their placement using force control combined with position-based visual servoing (PBVS). Visual servoing refers to a technique where feedback from the camera is used to dynamically adjust the robot's position in real time to ensure accuracy and good alignment during the assembly process.

An interesting usage of automation and robotics can also be found in solving a bin picking problem. The authors of [7] use a camera mounted on a linear frame with two laser sensors providing depth data to localise parts from a container (bin). Researchers decided to avoid using 3D vision, which would involve complex data processing and higher computational costs compared to using a 2D vision system.

2.1.2 Object localization in robotics vision

To localise objects in the camera's field of view, various approaches could be used including masking, template matching or deep learning algorithms such as YOLO.

Masking is a technique that takes advantage of colour differences between object and their background. It is a simple and resource efficient algorithm, however it is only effective in environments with stable lighting conditions. Additionally, it may struggle with detecting objects whose colour is similar to the background, so it can only be used in specific circumstances [8].

Another method used in vision system to distinguish object of interest from the background is template matching. This technique compares sections of an image to a reference template to locate object. It is effective when the object's appearance remains consistent, meaning lighting conditions are stable and the object is not rotated or tilted as template matching struggles with detecting rotated objects [9].

A very common method used in automation for reliable object detection and localization is using YOLOv8 deep learning model [10]. Researchers from China Agricultural University [11] used it in combination with Kalman filtering and low-pass filter to detect, localise and grasp oysters from a conveyor belt. They achieved a high success rate of oyster pickup in static and dynamic settings. The limiting factor was conveyor belt speed which when increased, decreased the success rate of pickups. To improve detection and grasping accuracy researchers proposed:

1. Stabilizing the robot to reduce vibrations,
2. Mounting the camera higher above the conveyor belt to increase the field of view and allow for faster detection,
3. Moving the camera away from the robot to give it more time to adjust its position before grasping the oyster, reducing the number of unsuccessful grasps.

2.1.3 Visual Servoing and Feedback Control

Visual servoing has advanced in recent years. A review by Hashmoto [12] highlights the development of vision based control algorithms. He explains the two main methods used for visual servoing: image-based visual servoing (IBVS) and position-based visual servoing (PBVS). IBVS uses the image from the camera directly, which provides resilience against camera calibration errors, but may be problematic due to camera-object interactions. On the other hand, PBVS estimates the 3D pose of the target object in Cartesian space, which is more intuitive, but is dependent on accurate 3D modelling and camera calibration. Overall, visual servoing is especially useful for systems where the robot needs to adaptively change its position based on the target object.

This project was also inspired by researchers from the Automation and Control group at the Technical University of Denmark. They developed a system for picking and placing loins of meat from a conveyor belt onto designated racks [13]. Their system achieved a success rate of 85% and the failures were mostly caused by low friction between the gripper and the loin which was slipping. Their robotic process is divided into four stages: cruising, tracking, grasping, and hanging, which is an actual robot state division used in this project.

2.1.4 Prediction and Filtering Techniques for Object Tracking

The authors of [14] propose a visual servo control algorithm to estimate the position of a moving object. Instead of using position-based visual servoing (PBVS), which relies on precise models of the camera and the robot to calculate the actual coordinates of the object in relation to the robot's base, the authors propose an image-based visual servoing (IBVS) which controls the robot based solely on image plane information, avoiding the need to reconstruct 3D coordinates. This approach eliminates the errors from robot and

camera models, which could stem from camera calibration and hand-eye calibration; see section 5.3 for more details. To further improve the precision of their system, the authors used a *Kalman filter* to improve pose estimation. The results demonstrated that the incorrect data from a camera can be drastically reduced when using a Kalman filter to correct it.

In another work [15], the authors presented a novel approach of using an adaptive Kalman filter to estimate the position of a moving object. In contrast to a traditional Kalman filter, which relies on fixed noise covariance matrices, the adaptive Kalman filter estimates the measurement noise in real-time, making it ideal for environments with changing and unpredictable conditions. For example, where the conveyor speed changes or the lighting conditions vary in time. To achieve it, authors used fuzzy logic controllers to tune the Kalman filter in real-time. Those essentially work by measuring the discrepancy between camera output and predicted output and if the prediction quality degrades fuzzy rules adjust the noise covariance matrices to improve the prediction. The result authors got from implementing this method demonstrated significant improvement in tracking quality compared to a traditional Kalman filtering.

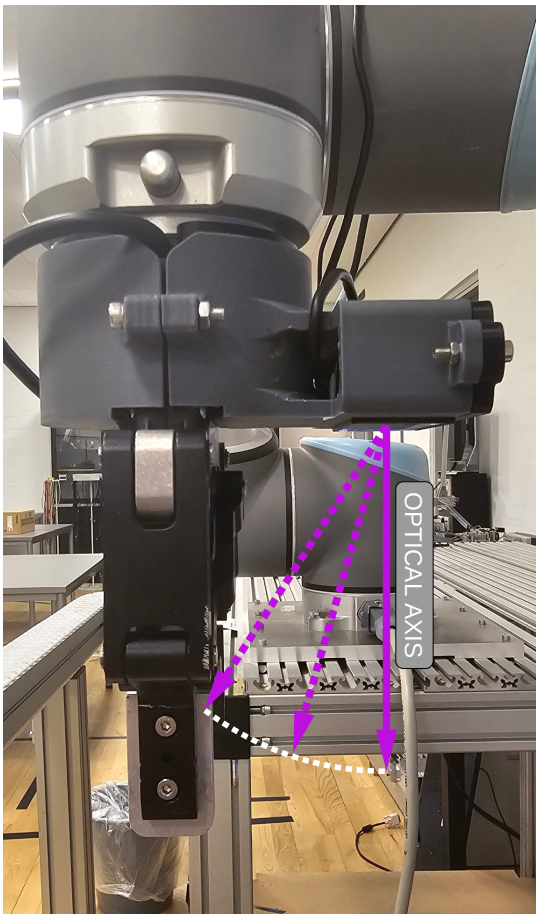
2.2 Decisions

This project focuses on the development of a robotic system that tracks and accurately grasps items from a conveyor belt.

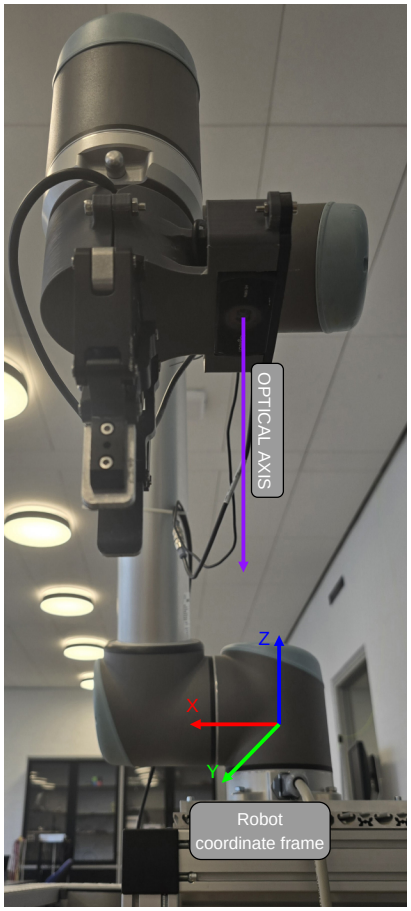
- **Product design** - simplified power supply used as a product handled by the robotic arm. More about this topic can be found in Chapter 3.
- **Camera mounting point** - When it comes to the vision system, it was decided that the camera will be mounted to the arm itself. This decision was guided by the fact that it would be very difficult to set up just one camera to completely cover the entirety of the relevant workspace while having enough resolution to accurately detect and pinpoint objects in the camera's field of view. Mounting the camera on the robotic arm allows the system to move the camera during the performance of each task, providing flexibility during part detection. This approach simplifies the hardware setup and eliminates the need for several fixed cameras.
- **Camera mounting tool** - the camera on the robotic arm could be mounted in many configurations. Figure 2.1a presents example variations in configurations with dashed lines that indicate alternative optical axes. The solid line represents the optical axis of a camera in a configuration used in this project. However, as described in the paper [16], if the optical axis of the camera is not aligned with the Z-axis, as shown in figure 2.1a, even a small image error can cause millimetre-scale position misalignments. This results from geometric transformations in the hand-eye transformation procedure, (see Section 5.3) and to avoid adding additional sources of error it was decided that the camera will be mounted with its optical axis parallel to the Z axis, as shown in figure 2.1b.

2.3 Summary

The literature overview provided valuable insights into key applications of vision-guided robotics in industrial settings. Several object detection methods were analysed including masking, template matching and deep learning methods. With this information, the actual detection technique for this project was chosen. Additionally, findings about visual servoing and feedback control as well as filtering techniques helped with deciding on the robot's control methods.



(a) Camera mount and alternative optical axis configurations



(b) Optical axis of the camera and robot coordinate frame

3

Product design: power supply

This chapter focuses on the design of the object that is handled by the robotic system.

3.1 Product design choice

After evaluating several options, at first a battery pack was chosen, because of its structured layout and relatively low complexity. This choice was inspired by the work presented in a paper [17], where researchers designed a robotic system for assembling batches of CR2023 cells. However, CR2023 cells were considered too small for this project's requirements, so a much larger 26650 cell was selected. The prototype design, based on the 26650 cell, is shown in figure 3.1b.

However, it turned out to be a poor choice, since the batteries were rolling off the conveyor belt. Moreover the conveyor belt used had a width of only 8cm, which made it impractical to design a small tray or fixture to hold batteries safely during the movement of the conveyor belt. Because of those limitations, this prototype was discarded and other product ideas were considered.

The second idea was to design a simplified smoke detector 3.1a. The concept was inspired by a real-world solution developed by *Lyric Robotics*[1]. The smoke detector prototype was less prone to shifting or rolling during the robot handling process, because of its flat shape. However, it was discarded, because the round design turned out to be troublesome. Its circular shape meant that the gripper's fingers would have to be redesigned to securely handle it. This added complexity led to the decision to choose a more well-shaped object for the robot to manipulate.

The third design idea was a power supply. The design choice was proposed collaboratively by the thesis promoter and the author. Several prototypes were designed and tested. They were easy to detect by the vision system, because of their uniform rectangular shape. Rectangles have well-defined edges and corners making them straightforward for computer vision algorithms to identify and localise. Additionally, two out of three parts of the designed power supply did not cast any shadows, which is very helpful for the vision system. It is well known that shadows in vision systems and robotics, often causing errors in object detection. Even though various methods to eliminate them were developed to

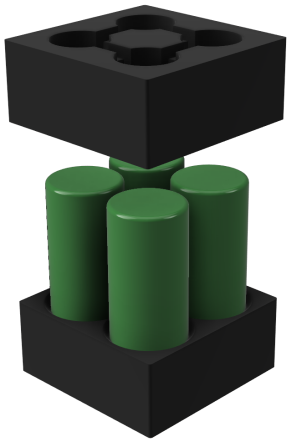
[1] https://en.lyric-robot.com/solution-5-12-1.html

address this issue, shadows still are a challenge in vision systems [18]. Each part of the designed power supply could also be easily grabbed by the robotic gripper. Considering all the advantages of this design including ease of manipulation and compatibility with vision system, the power supply was chosen as the final product.

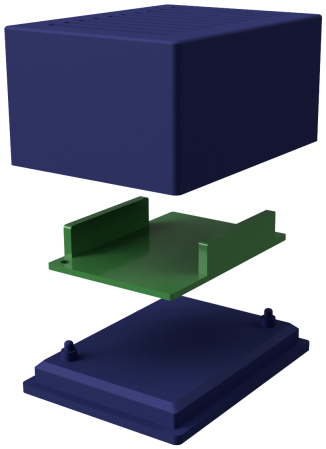
3.2 Design

This power supply consists of three parts: the top casing, bottom casing and integrated circuit board. The top casing includes grills for simulating ventilation slots similar to those found in a real power supplies. The bottom casing contains a press-fit slot and includes internal pins for mounting the dummy integrated circuit board. The integrated circuit is not based on a real design, it is simplified and only includes two vertical walls meant to represent heat sinks. Those heat sink structures are also to help the robotic arm to grab and manipulate the integrated circuit board. All of the parts were designed in Fusion 360 and were 3D printed using different PLA colors to enhance visual distinction by the camera.

(a) Smoke detector in Fusion 360



(b) Battery pack in Fusion 360



(c) Power supply in Fusion 360

4

Robotic system architecture

This chapter overviews the technical setup used in this project, along with corresponding drivers and communication interfaces used to control each device. The system consists of a UR5 robotic arm, a Robotiq 2F-85 gripper, an X85 Plastic Chain Conveyor, WA20 DR63M4 motor with inverter, and a camera mounted to the end effector of the robotic arm, which will be described in the next chapter dedicated to vision (see Chapter 5).

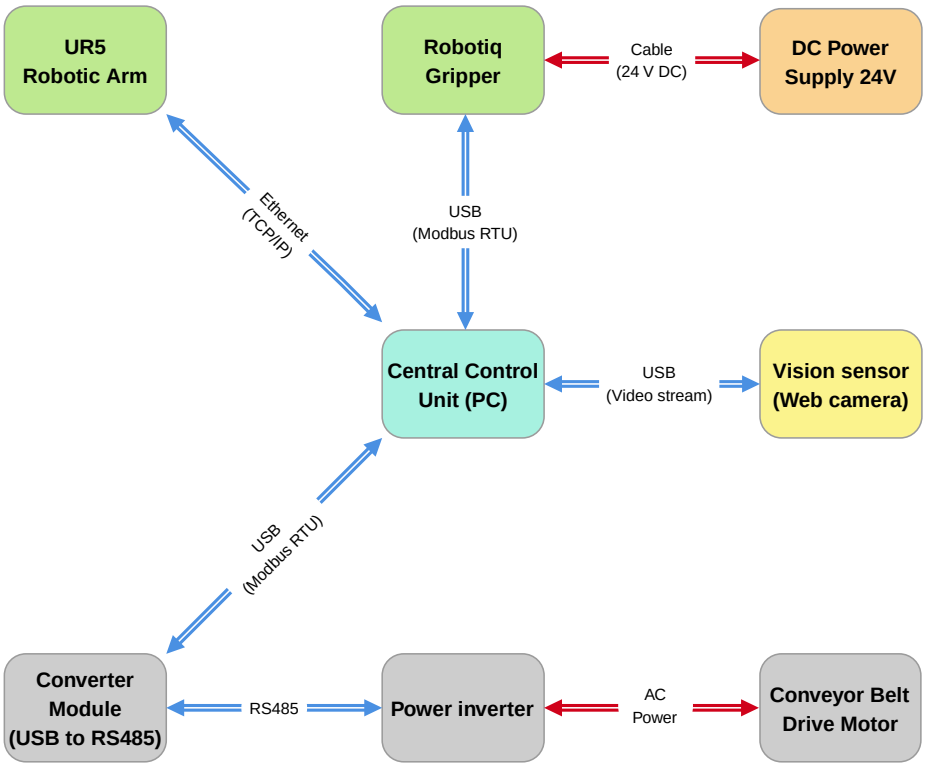


Figure 4.1: Robotic system architecture

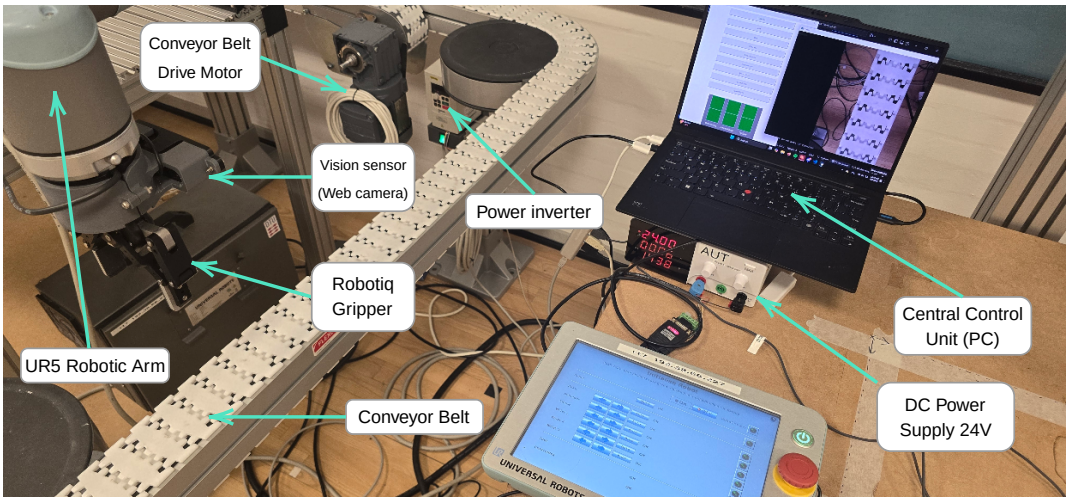


Figure 4.2: Robotic system setup

In the figure 4.2 the converter module is mounted under the conveyor belt and is not visible in the picture.

4.1 Robotic arm

This project uses the Universal Robots UR5 robotic arm. It's a collaborative robot designed to work safely alongside humans in a shared workspace. It is built with features to reduce the risk of injury and allow the user to directly interact with the robot - for example, by physically teaching it a specific pose [19]. It has 6 degrees of freedom and a reach of 850 mm, which allows the robot to cover a large portion of the workspace.

4.1.1 Robot motion control driver

The first driver manages the movement of the robotic arm by sending commands in accordance with the URScript programming language described in the documentation *Script Manual*[1]. The commands are sent via TCP/IP over port 30001, which is a dedicated port for robot control. More details about the UR communication interfaces can be found on the Universal Robots website: *Overview of Client Interfaces*[2].

Communication with the robot in Python is set up using the `socket` library, through which the commands are sent encoded in UTF-8. An example command is shown bellow:

```
movel(p[0.15, 0.766, 0.081, 3.1415, 0, 0], 0.05, 0.1, 3)\n
```

This instruction commands the robot to move to a specific position `p[0.15, 0.766, 0.081, 3.1415, 0, 0]` with an acceleration of $0.05\,m/s^2$ and maximum velocity of $0.1\,m/s$ within a 3-second time-frame.

The driver only includes the most crucial commands for the robot to operate within the scope of this project. Additionally, all errors and information are displayed in the terminal with the help of the `logging` library.

[1] https://s3-eu-west-1.amazonaws.com/ur-support-site/115824/scriptManual_SW5.11.pdf
[2] https://www.universal-robots.com/articles/ur/interface-communication/overview-of-client-interfaces/

4.1.2 Real-time position read driver

The second driver retrieves a real-time position of the robotic arm. It connects to the robot through port 30003 [2] which is used as a real-time interface for the robot. The driver uses the `socket` library used to continuously receive binary packets from the robotic arm.

This driver must run from the start of the program continuously to get the actual TCP position of the robot in real time, therefore it runs in a background thread that listens for incoming data and provides updates when needed. The driver also has a reconnection function to handle interruptions or disconnections from the robot.

To get the position of the robotic arm, the program only needs to call `URReceiver.get_pose()`.

4.2 Conveyor belt

To transport the parts to the robot workspace, an X85 Plastic Chain Conveyor is used. It is driven by a WA20 DR63M4 motor and is controlled by a VEVOR H100 inverter which controls the motor by setting specific frequency. It's set up in a looped configuration, so the parts not picked by the robot can circulate around it. In this way unpicked parts can return for further attempts at grabbing them by the robot.

4.2.1 Driver functionality

The conveyor belt is controlled by a custom driver that uses Modbus RTU communication standard. The Python implementation uses `ModbusSerialClient` library from the `pymodbus` library. To control the inverter specific values are written into registers as described in the documentation provided by the manufacturer Vevor documentation[3]. The driver enables control over the basic behaviour of the conveyor, including starting, stopping, direction control, and speed adjustment.

4.3 Gripper

This project uses the Robotiq 2F-85, a two finger gripper. The "2F" designation indicates its two-finger configuration, while "85" refers to the maximum opening width.

4.3.1 Driver functionality

The gripper is controlled with a custom driver and handles the communication with the device using the Modbus RTU protocol over an RS-485 physical interface, as described in the Robotiq 2F-85 documentation[4]

The driver includes functionalities such as connecting to the gripper, activating it, checking its status, and control over its finger position, speed, and force.

[3] https://images.thdstatic.com/catalog/pdfImages/b0/b008b752-3e4a-4c5c-aa62-3f425c50eff9.pdf

[4] https://assets.robotiq.com/website-assets/support_documents/document/2F-85_2F-140_Instruction_Manual_e-Series_PDF_20190206.pdf

5

Vision: camera and algorithms

Vision systems are crucial in the robotic industry. They are often used in collaboration with robotic arms to guide their movement and interact with moving elements. Beyond positioning, vision systems are also used to identify and categorize objects. In some other industrial applications, they are also used to detect flaws, imperfections, or damaged parts. Each section related to coding and algorithms represent separate modules in the system, they can be exchanged or altered without impacting other modules.

5.1 Related work

Substantial part of the vision algorithms and methodology applied in this project were presented in the DTU course 34759 Introduction to Robot Vision [20].

The approach used in the work [21] serves as a foundation for this project. The setup in this paper resembles the setup used in this project; the camera is mounted near the robot's grippers and is oriented straight down. This paper also describes the method used for calibrating robotic arm with the camera and how the coordinates of an object in real world are transformed to the coordinates for the robotic arm.

5.2 Camera Calibration

5.2.1 Theoretical Background of Camera Calibration

Camera calibration is a fundamental process in computer vision and robotics. This process determines internal parameters (intrinsic parameters) of a camera and external parameters. Calibration allows for the transformation of 3D points in the real world into the corresponding 2D coordinates in an image. Moreover, if the camera parameters are estimated correctly it enables the removal of lens distortions, which is mandatory for creating an accurate vision system, as shown in the work [22].

- Focal length (f_x, f_y) - describes the distance between the camera's optical center and the image plane (lens), when the lens is focused to infinity.
- Optical center (c_x, c_y) - is a point where the camera's optical axis intersects the image plane (lens). In a perfect scenario, it would be exactly in the middle of the lens, but due to imperfections and deviations, it is slightly displaced.

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

[23] [24]

- Distortion coefficients [23] [24]
  - Radial Distortion Coefficients - those coefficients are responsible for bulging or pinching effect of the image. The further from the centre of the image the point lines, the more severe the distortion becomes. It is caused by the lens bending the light unequally across it's surface. If the values of the radial distortion coefficients are known it is possible can reduce or completely correct radial distortion.
  - Tangential Distortion Coefficients - those coefficients are responsible for twisting or asymmetrical shifting of the image. The further from the centre of the image the point lines, the more severe the points are displaced. It occurs when the lens is not perfectly aligned with the camera sensor. If the values of the radial distortion coefficients are known it is possible can reduce or completely correct radial distortion.

$$\text{distortion_coefficients} = \begin{bmatrix} k_1 & k_2 & p_1 & p_2 & k_3 \end{bmatrix}$$

- Extrinsic parameters [24]
  - Rotation Matrix - describes the rotations around x, y, z axis.
  - Translation Vector - specifies the translation of the camera with x, y, z coordinates.

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

$$T = \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}$$

5.2.2 Camera calibration procedure

The camera calibration procedure in this study was performed by using the chequerboard method. This method is widely used due to it's simplicity and effectiveness. It relies on capturing multiple images featuring a chequerboard pattern **??** using the desired camera that needs to be calibrated. [25]

Using the camera mounted to a robotic arm, a series of 40 pictures has been taken. Each one has a clear view of the chequerboard and was taken with the chequerboard in various orientations, distances, and angles. It's crucial to have diverse pictures to improve the results of the calibration process. An example picture taken by the camera is displayed below 5.1.

To take the pictures of the chequerboard in different orientations and positions, two robotic arms were used. The first one had a camera mounted to its effector, and the second one

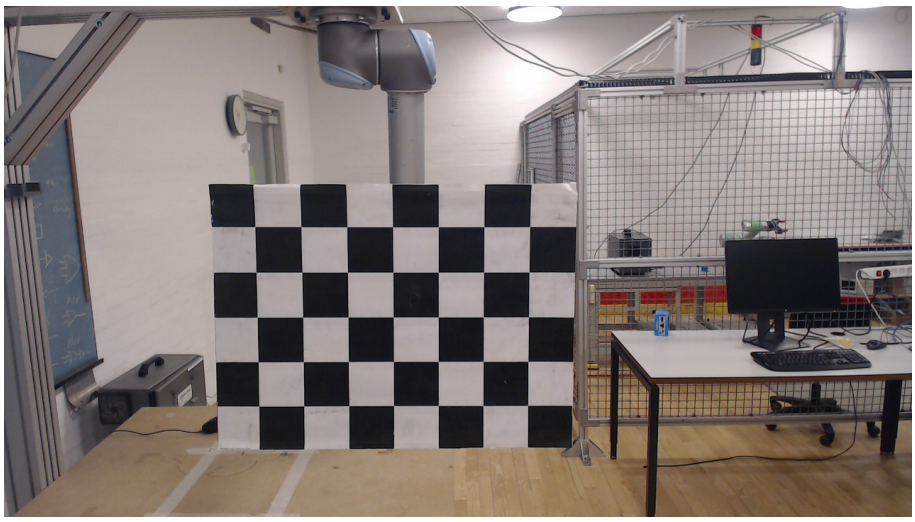


Figure 5.1: An example picture with a chequerboard

held a plywood board with a chequerboard attached to it. By moving the arm that had a chequerboard pattern, 40 pictures had been taken. Each showing a chequerboard from unique distance, angle and pose.

The images are taken using a simple python script. This script displayed a live feed from the camera on the computer screen and allowed the user to press the 'q' key and capture images when the desired chequerboard position was reached. 40 pictures with a resolution of 1280x720 were taken. This resolution was selected because it was sufficient for this application, and higher resolutions were too big for the computer screen to display properly. A simplified snippet, as shown in Figure **??**, explains how the program works. Each image was saved in a designated folder and assigned a unique filename based on a sequential numbering. This set of images is enough to precisely calculate intrinsic and extrinsic parameters of the camera. According to Zhang [25], capturing images from just two or three different angles is enough to yield results, although increasing the number of pictures improves the accuracy and precision of the calculated parameters.

When the set of pictures was completed a calibration procedure could be started. This was done by running a python script that analysed each picture to detect a chequerboard pattern. This process also involved doing a series of image preprocessing methods to ensure accurate corner detection. The full overview of operations is explained here:

1. **Reading the image:** Loading the image.
2. **Gray-scale conversion:** Converting the image to gray scale, it simplifies the algorithm and is widely used practise in computer vision. Additionally, since chequerboards are black and white keeping colours is unnecessary.
3. **De-noising:** Using Open-CV's de-noising function noise is reduced while keeping the edges. This method improved corner detection and accuracy.
4. **Sharpening:** Sharpening the the image containing the chequerboards to enhance edge contrast and make the corners more distinguishable.
5. **Histogram equalization:** Used to enhance the contrast. It's especially useful when lighting conditions are uneven or where the image appears too dark or too bright.

6. **Chequerboard detection:** The chequerboard corners were detected using OpenCV function, which searches for a specified grid pattern within the image.
7. **Refining corners:** If the chequerboard was successfully found, the corner positions was refined to improve the accuracy of placement the detected corners.
8. **Drawing corners:** At the refined corners were drawn on the image as shown on picture 5.2

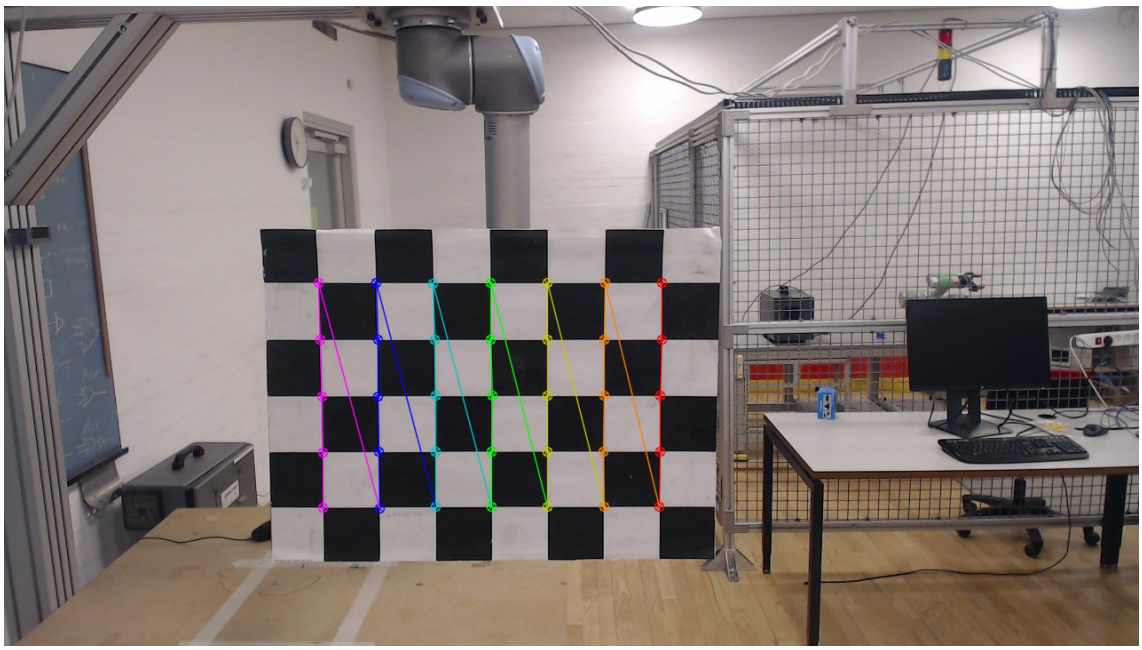


Figure 5.2: An example picture with marked chequerboard corners

After all the pictures were analysed for the chequerboards the intrinsic and excentric parameters of the camera were calculated. The results are listed below:

```
Camera Matrix:
[[911.49,      0, 616.60],
 [     0, 911.61, 348.08],
 [0     ,      0,      1]]

Distortion Coefficients:
[ 0.1020, -0.2222, -0.0014,  0.0012,  0.0571]
```

The results indicate that the focal lengths are approximately 911.49 in *x* direction and 911.61 pixels in *y* direction, respectively. The optical centre is located at pixel coordinates (616.60, 348.08), which is close to the centre of the image. The image centre is located at pixel coordinates (640, 360).

The distortion coefficients consist of five values:

- $k_1 = 0.1020$, $k_2 = -0.2222$, and $k_3 = 0.0571$ are the **radial distortion coefficients**, which account for barrel or pincushion distortion effects.
- $p_1 = -0.0014$ and $p_2 = 0.0012$ are the **tangential distortion coefficients**, which correct for misalignment between the lens and the image sensor. In this case they are very low. Which indicates minimal tangential distortion.

5.3 Hand eye calibration

In robotic systems, calibrating a robotic arm with a camera is known as hand-eye calibration. This process establishes a spatial relationship between the robotic gripper (hand) and a camera (eye). When the calibration is done correctly, it allows for the robot to effectively cooperate with the camera. In this project, the camera is mounted to the robotic end effector with a custom 3D printed bracket, as shown in the picture 5.4.

The optical centre of the camera is offset 8 cm from the centre of the gripper and is taken into account in the calibration process.

There are two primary methods of setting the system for vision-based robot control as described in a paper [26].

- **Hand-eye calibration** - is used when the camera is mounted to the robotic arm and moves with the robot. It involves finding a transformation from the robot tool frame to the camera frame.
- **Robot-world-hand-eye calibration** - is used when the camera is set stationary in the robot's work space. It involved finding a transformation from the robot base or robot tool frame to the camera.

Those two approaches differ in a mathematical description and the number of transformations. In this project, a hand-eye calibration was chosen, because the camera is mounted to the robot's end effector. As proposed by the method used by Y.C. Shiu, S. Ahmad [27] the hand-eye calibration involves solving the homogeneous transform equation:

$$A_{c2}^{c1} X_h^c = X_h^c B_{h2}^{h1} \tag{5.1}$$

where:

A_{c2}^{c1} : Transformation between two camera poses
B_{h2}^{h1} : Transformation between two robot poses
X_h^c : Transformation from the robot tool frame to the camera frame

In the notation used throughout this work, the superscripts and subscripts denote coordinate frames. For example A_{c2}^{c1} refers to a transformation between camera pose 1 and camera pose 2; similarly, B_{h2}^{h1} refers to a transformation between robot pose 1 and robot pose 2. Finally, X_h^c stands for the fixed transformation from the robot's tool coordinate frame h to the camera coordinate frame c.

The process of doing the hand-eye calibration is similar to the procedure used to calibrate the camera and remove distortions as shown in section 5.2. The hand-eye calibration method applied in this project is described in detail in paper [21] and [28]. The main steps involve the following:

1. Calibrating the camera - calculating the extrinsic and intrinsic parameters.
2. Capturing chequerboard pictures - mounting the camera to robots end-effector and moving it to various poses while capturing pictures and saving corresponding poses of the robotic arm while doing so.
3. Estimating the pose of chequerboard - using Open-CV's pose estimation functions determine the pose of the chequerboard relative to the camera.
4. Solving hand-eye transformation - using the robot and camera poses to solve the hand-eye transformation.

After following those steps the algorithm determines the transformation between the camera coordinate frame to the robot's end-effector frame expressed as a rotation matrix **R** and translation vector **t**:

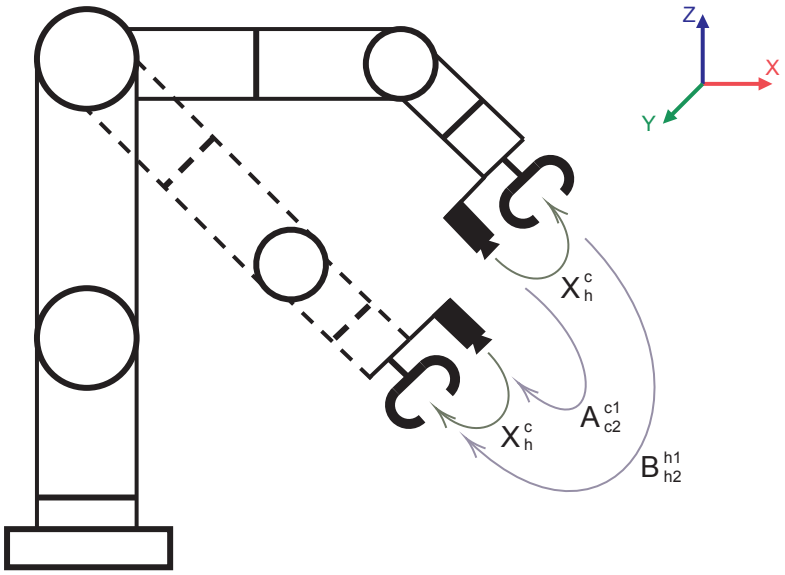


Figure 5.3: Setup for hand-eye calibration

$$\mathbf{R} = \begin{bmatrix} 0.0025 & 0.99998 & 0.0065 \\ -0.99995 & 0.0026 & -0.0093 \\ -0.0093 & -0.0064 & 0.99994 \end{bmatrix}, \qquad \mathbf{t} = \begin{bmatrix} -0.0592 \\ -0.0140 \\ -0.1149 \end{bmatrix}$$

To validate if the results from the hand-eye calibration are correct, the transformation results can be compared with the physical placement of the camera. First, the rotation matrix **R** represents the rotation of the camera frame relative to the robot's end-effector tool frame. Each row of this matrix defines how one of the camera's axes aligns with the robot's end-effector coordinate system.

- **First row** – Camera's **X-axis** in tool frame:

  $$\vec{x}_{\text{cam}} = 0.0025 \cdot \hat{x}_{\text{tool}} + 0.99998 \cdot \hat{y}_{\text{tool}} + 0.0065 \cdot \hat{z}_{\text{tool}}$$

  This indicates that the camera's X-axis is almost perfectly aligned with the robot's Y-axis.

  The small components along the X and Z-axis corresponds to a tilt of approximately $\theta \approx \sin^{-1}(0.0065) \approx 0.37°$ and $\theta \approx \sin^{-1}(0.0025) \approx 0.14°$. Those values are negligible and for the purposes of this project can be approximated to zero in practice.

- **Second row** – Camera's **Y-axis** in tool frame:

  $$\vec{y}_{\text{cam}} = -0.99995 \cdot \hat{x}_{\text{tool}} + 0.0026 \cdot \hat{y}_{\text{tool}} - 0.0093 \cdot \hat{z}_{\text{tool}}$$

  This indicates that the camera's Y-axis points almost perfectly in the negative X direction of the robot's frame. The minor Y and Z-axis corresponds to a tilt of approximately $\theta \approx \sin^{-1}(0.0026) \approx 0.15°$ and $\theta \approx \sin^{-1}(-0.0093) \approx -0.53°$. Those values are negligible and for the purposes of this project can be approximated to zero in practice.

- **Third row** – Camera's **Z-axis** in tool frame:

  $$\vec{z}_{\text{cam}} = -0.0093 \cdot \hat{x}_{\text{tool}} - 0.0064 \cdot \hat{y}_{\text{tool}} + 0.99994 \cdot \hat{z}_{\text{tool}}$$

This indicates that the camera's Z-axis (optical axis) is very closely aligned with the robot's Z-axis.

The minor X and Y-axis corresponds to a tilt of approximately $\theta \approx \sin^{-1}(-0.0093) \approx -0.53°$ and $\theta \approx \sin^{-1}(-0.0064) \approx -0.37°$. Those values are negligible and for the purposes of this project can be approximated to zero in practice.

As for the translation vector, it describes an offset from the centre of the robot's tool frame to the optical centre of the camera. The values indicate that there is an offset of approximately -5.92 cm along the X-axis, -1.40 cm along the Y-axis and -11.49 cm along the Z-axis. These values were validated against real-world measurements and closely match the actual physical configuration.

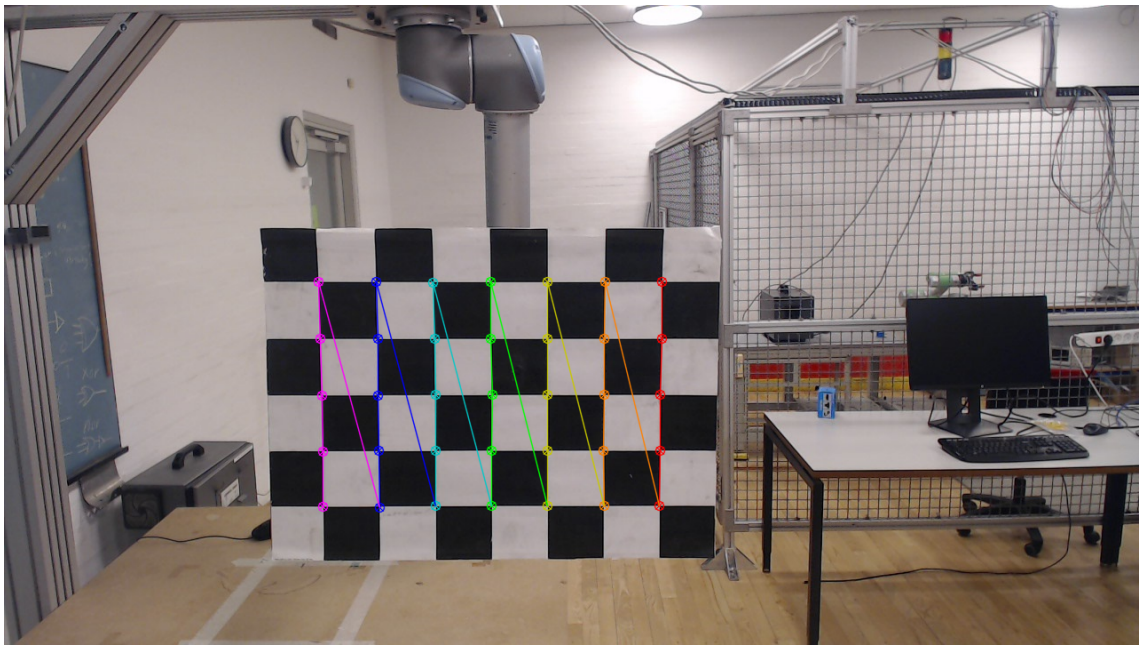


Figure 5.4: Camera mounted to the robotic arm

5.4 Vision system methodology

The project's main loop chart is presented on a figure 5.5. Vision system in this project is responsible for two main functionalities:

1. **Part classification and angle estimation:** Estimating the angle of rotation and determining the specific part of the power supply based on the area of the object seen by the camera.
2. **Center pose tracking:** Calculating the coordinates of the part's centre relative to the end effector.

Although those two functions differ in functionality they both share the same object contour detection pipeline which is overviewed in the following section.

5.5 Image processing pipeline

After completing the camera calibration and hand-eye calibration, the image processing pipeline was developed. The main goal is to separate the object of interest from the background using a mask-based method. Each frame captured by the camera is processed

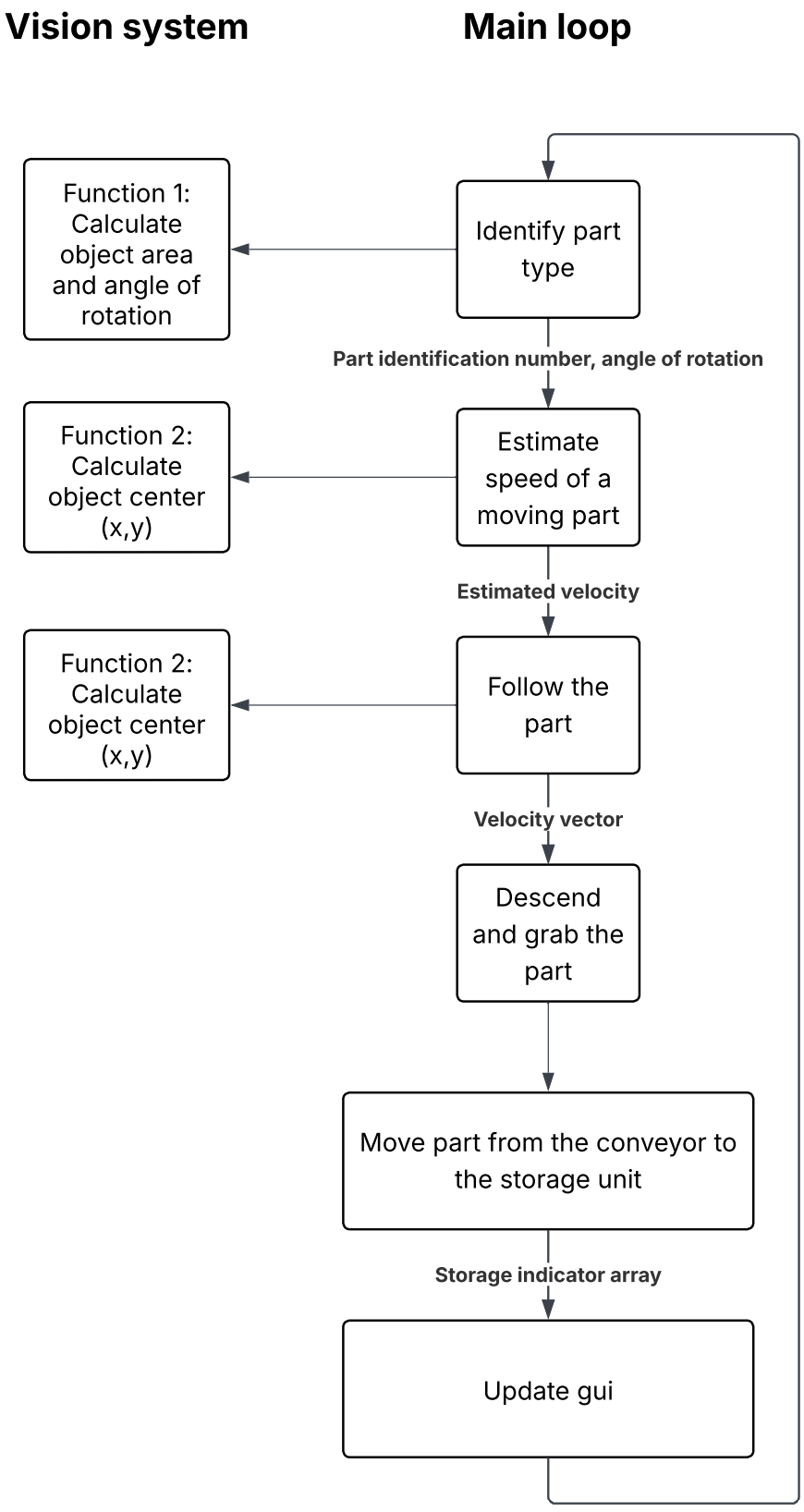


Figure 5.5: Overview of the system's main control loop

according to specific parameters. Each part of the power supply has a predefined set of parameters tailored to it.

The processing pipeline includes the following steps:

1. **Undistorting and cropping the image** — The raw image from the camera is first corrected for lens distortion and cropped according to the ROI[1] (region of interest).
2. **Reducing the region of interest** — The ROI is further narrowed by placing two black rectangles on the sides of the frame. This method avoids shifting the image coordinates, which would happen if the image was cropped as a whole.
3. **Parameter assignment** — Depending on the detected part, a corresponding JSON[2] file is loaded. Parameters such as HSV[3] colour bounds, as well as dilation[4] and erosion[4] iteration counts, are extracted and assigned to variables.
4. **Contrast enhancement** — To improve the contrast of an image, particularly under poor lighting conditions, CLAHE[5] (Contrast Limited Adaptive Histogram Equalization) is used. The image is converted to LAB[3] colour space (L — lightness, A — green-red component, B — blue-yellow) to modify the lightness channel specifically, keeping the others unchanged. LAB colour space is chosen because it separates luminance from chromaticity without affecting colour.
5. **Colour space conversion** — The image is converted from LAB space to HSV space. HSV is used to extract specific colour ranges from the image, where H stands for hue, S for saturation, and V for value. After converting to HSV, a mask is applied to extract only pixels within the specified colour range.
6. **Creating a mask** - Using the HSV threshold values from the JSON file a binary mask is created.
7. **Initial noise reduction** — Morphological operations[6] of closing and opening are performed to reduce noise, fill small holes and remove small artifacts from the mask.
8. **Mask application** - The mask is applied to extract regions of specific colours.
9. **Further noise reduction** — To further refine the result, dilation and erosion operations are applied. Their strength depends on the parameters loaded from the JSON file.
10. **Contour detection** — Finally, valid contours are extracted from the image. Canny[7] edge detection identifies sharp transitions corresponding to object boundaries. After applying the Canny filter, contours are extracted and filtered according to their area.

The HSV colour parameters, dilation and erosion iteration counts, are calibrated using a dedicated window in the GUI (see Section 5.8) of the system.

The image processing pipeline represents a separate module, allowing it to be easily altered or switched for another method if needed. An alternative method to the one used in

[1] https://docs.opencv.org/4.x/d3/df2/tutorial_py_basic_ops.html
[2] https://www.w3schools.com/python/python_json.asp
[3] https://opencv.org/blog/color-spaces-in-opencv/#h-hsv-hue-saturation-value-color-space
[4] https://docs.opencv.org/3.4/db/df6/tutorial_erosion_dilatation.html
[5] https://docs.opencv.org/4.x/d5/daf/tutorial_py_histogram_equalization.html
[6] https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html
[7] https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html

this project would be a machine learning algorithm such as YOLO used by the researchers from China Agricultural University described in section 2.1.2.

In another study [29], researchers from the University of California developed a Grasp Quality Convolutional Neural Network (GQ-CNN) to predict grasp success. Their system was trained on a large dataset of point clouds and grasps. In their experimental setup they achieved a high precision of 99% and a faster and more reliable response compared to the traditional methods.

However, those solutions require big sets of training data and good computational resources. In contrast, the approach used in this project is lightweight and simpler than using machine learning methods. Moreover, both of those projects dealt with complex objects of varying shapes, sizes and colours. If objects of interest are more uniform in shape, colour and dimensions, a simpler algorithm such as masking can be used as in this project. A study from Chungnam National University [30] proposed a similar solution to the one used here. They use background subtraction and HSV masking to detect objects in the camera's field of view. Unlike the manual setting of HSV parameters employed in this work, their method determines those based on random sampling of the region of interest (ROI), HSV similarity comparison and ROI merging operations.

5.6 Part classification and angle estimation

Before guiding the robot to grab an object moving on the conveyor belt, it is crucial to determine which part of the power supply is currently under the robot's end effector. The identified part number determines which JSON configuration file is loaded for the masking algorithm. If an incorrect file is selected, the image processing pipeline will fail to correctly identify the object's contour.

Moreover, the part number is used during the sorting phase, where each part is placed in a specific location within a 3D-printed part sorter. The part numbering used in this project is shown in figure 5.6.

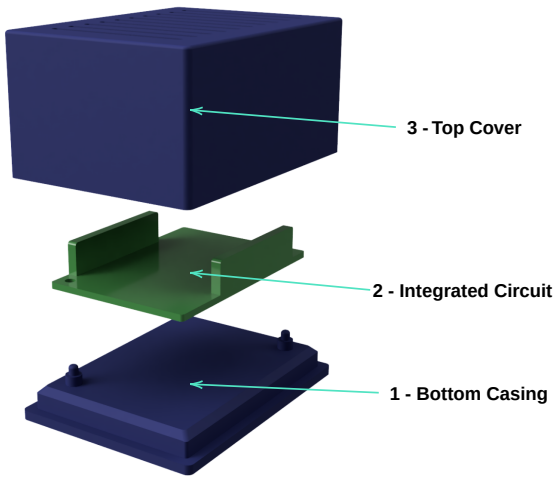


Figure 5.6: Power supply part numbering

The classification algorithm works based on the area of the contour of the part in the camera field of view. The method used cycles rapidly between available part numbers (in this case from 1 to 3). For each part number an appropriate JSON configuration file is loaded, which contains HSV thresholds and morphological parameters for the masking

algorithm (see Section:5.5). If the area of the detected contour is larger than fixed threshold of 20000 pixels that part number is considered a match. However, this part number is preliminary and is further checked to ensure that the algorithm is robust.

Once the preliminary part number is found, it is further checked by capturing three valid areas and angles of the part. If it fails to do so within a 15-second timeout the part number is considered invalid and the algorithm terminates and displays an appropriate message in the terminal.

If the algorithm manages to collect three valid (non-None and non-empty) area and angle values, the data is then filtered. Afterwards, the average of both is calculated and the final part number is selected based on the averaged area.

Although this algorithm is relatively simple compared to other alternative methods such as machine learning, it performs reliably and has proven to be quite effective with no mismatches observed during the testing phase.

The flowchart illustrating the operation of the function responsible for part classification and angle estimation is as shown in Figure 5.7.

5.7 Center pose tracking

For the robotic arm control system to function effectively it must be updated with real-time information about the pose of objects moving on the conveyor belt. This information can be determined through processing images captured by the camera mounted to the end-effector of the robotic arm. The vision processing pipeline described in subsection 5.5 provides object contours from the images captured by the camera, and in this section, it will be overviewed how from those contours the real-world position is calculated.

By using this approach it's possible to develop an efficient 3D localization procedure with using only 2D images. This eliminates the need for dedicated depth sensors and reduces the system complexity and cost.

The output of the localisation function is not expressed as global or world coordinates, but in a coordinate system that's relative to the end-effector. For example, if the algorithm returns a position of [0.10, 0.00, 0.00] it indicates that the object is located 10 cm in the positive X direction with respect to the end-effector frame. This method facilitates the implementation of control algorithms that rely on calculating speed vectors from position vectors.

5.7.1 Center pose tracking overview

The pinhole camera model describes the projection of a 3D point onto a 2D image plane (see Figure 5.8). This models shows how a 3D point (X, Y, Z) is projected at coordinates (u, v) onto the image plane, using intrinsic camera parameters and known camera geometry. This subsequent algorithm for pose estimation is based on this model.

The algorithm in this section uses contours provided by the function from subsection 5.5 to estimate object coordinates relative to the robotics arm end-effector. This function provides multiple contours, so this algorithm only chooses the contour with the maximal area. This assumption is valid, because if the masking is done correctly contour with the maximal area is the part right under the camera on the conveyor belt.

After selecting the contour with the maximal area a minimum-area bounding rectangle is fitted around it. This step provides x and y coordinates of the part's centre as well as it's angle of rotation. Additionally, after this step four corner points of the rectangle

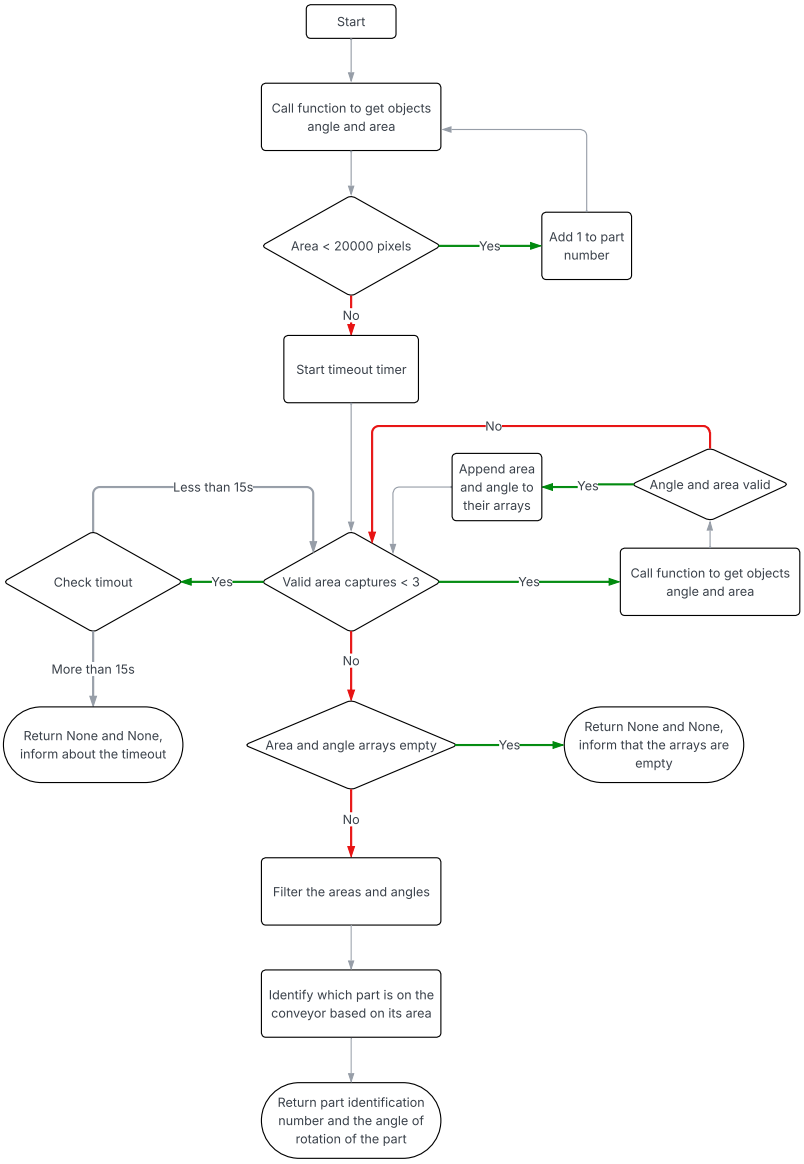


Figure 5.7: Part classification and angle estimation function

are calculated and sorted to ensure their stability between frames. This is necessary, because the raw order of the points can vary significantly between frames, which would lead to instability in visualisation or potential problems in the later calculations. The sorting is done by computing the angle between each corner point and the centre of the rectangle. And arranging them in the next step anti-clockwise order.

However, in this algorithm, the corner points are used only for visualization and not for any calculations relevant to the pose estimation. Since the distance to the object in the picture is a known variable coordinates of the centre point are sufficient to estimate pose of the object in the picture. If this distance would be unknown, it would be necessary to use an algorithm such as PnP (Perspective-n-Point) to estimate the actual position of the object. The benefit of the known distance is that it requires less computation and the estimated coordinates are more accurate.

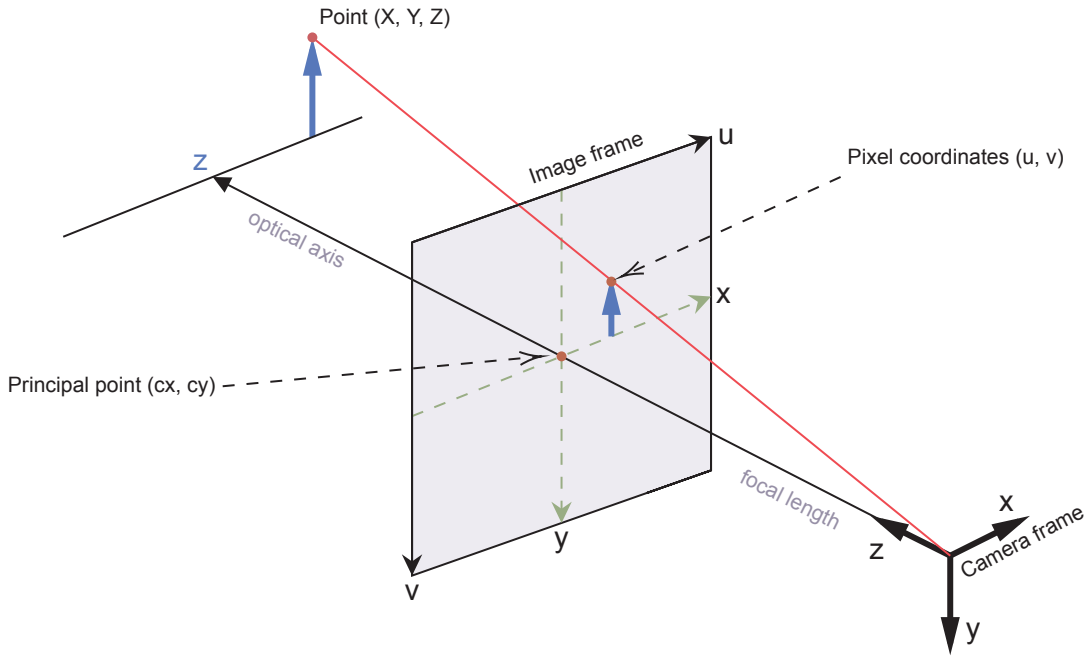


Figure 5.8: Pinhole camera model

Given a detected 2D pixel point (u, v) in the image, the following steps were taken to convert 2D pixel point to a 3D point in the robot coordinate frame:

1. Convert 2D pixel coordinates into a homogeneous pixel coordinates.

$$\mathbf{p}_{\text{pixel}} = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

2. Back-project to Normalized Camera Coordinates using camera matrix K from camera calibration procedure (see Section:5.2). Here, $\mathbf{d}_{\text{cam}}$ is a direction vector in the camera coordinate system.

$$\mathbf{d}_{\text{cam}} = K^{-1} \cdot \mathbf{p}_{\text{pixel}} = \begin{bmatrix} x_n \\ y_n \\ 1 \end{bmatrix}$$

3. Scale direction vector $\mathbf{d}_{\text{cam}}$ by the known depth Z. It's a known value, because for this project the robot is always keeping the same distance to the conveyor belt.

$$\mathbf{P}_{\text{cam}} = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} x_n \cdot Z \\ y_n \cdot Z \\ Z \end{bmatrix}$$

4. Transformation to the robot coordinate frame using parameters from hand-eye calibration (see Section:5.3).
   - $R_{\text{cam}\to\text{gripper}} \in R^{3\times3} \to$ rotation matrix from camera to robot frame.
   - $t_{\text{cam}\to\text{gripper}} \in R^{3\times1} \to$ translation vector from camera to robot frame.

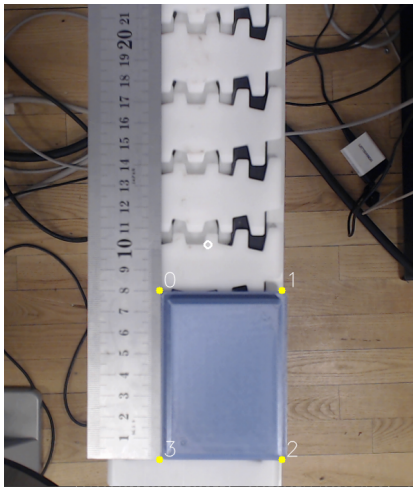
Final 3D coordinates:

$$\mathbf{P}_{\text{robot}} = \mathbf{t}_{\text{cam}\rightarrow\text{gripper}} - \mathbf{R}_{\text{cam}\rightarrow\text{gripper}} \cdot \mathbf{P}_{\text{cam}}$$

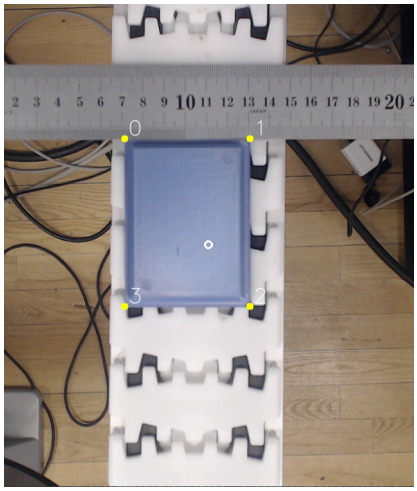
This algorithm was developed with the help of OpenCV documentation [31]. To verify the accuracy of the algorithm as simple test was performed. A bottom part of the 3D printed power supply part was placed on a stationary conveyor belt with a ruler as a reference point. The test was done separately for x and y positions. Figure 5.9 shows the camera view during the testing.

The first column shows the testing of the X position estimation. In the top image the part is positioned at 0.0 m according to the camera system. The part was then shifted by -0.05 m and -0.1 m and the camera measured -0.049 m and 0.099 m respectively. In both cases, the measurement error was approximately 1 mm. This deviation is minimal and shows that the X coordinate estimation is accurate.

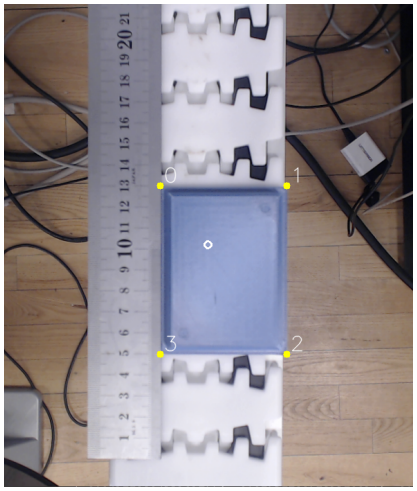
The second column presents the test results of the Y position estimation. The top image shows the part positioned at 0.0 m according to the camera system. The part was then shifted by -0.02 m and 0.02 m along the Y-axis and the camera measured the -0.019 m and +0.02 m respectively. The measurement error was approximately 1 mm in the negative Y direction and in the positive was negligible, indicating that the Y coordinate estimation is accurate.



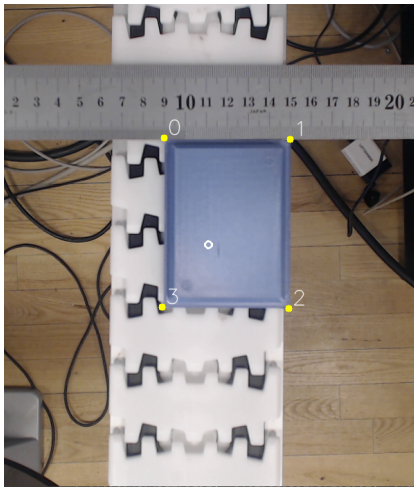
X position: 0.000 m



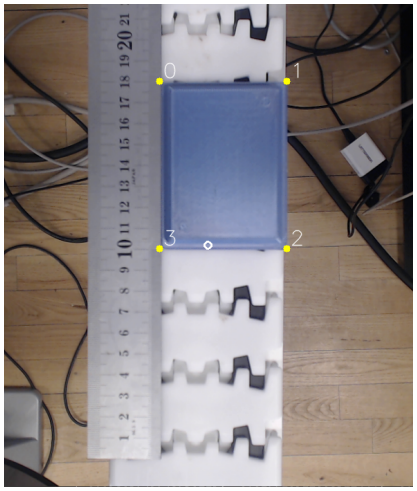
Y position: 0.000 m



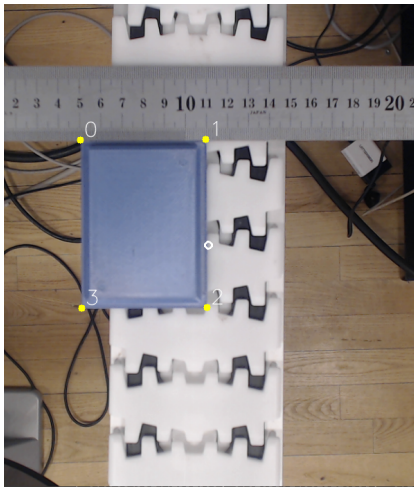
X position: -0.049 m



Y position: -0.019 m



X position: -0.099 m



Y position: +0.020 m

Figure 5.9: Camera view of part positions during testing, clipped to the relevant region. Left column shows variations in X position and right column shows variations in Y position.

5.8 Graphical user interface (GUI)

Graphical interfaces in this system is used to interact with the system. This project is not meant for external users, so it's not crucial to make it user-friendly. Functionality of GUI:

- Camera view - live view from the camera mounted to the robotic arm displaying additional information from image processing pipeline, such as contour lines, corners and corner numbering. Allows for the user to assess the performance of the system.
- Calibration - if needed a possibility to perform a calibration of parameters used for image processing.
- Storage status - simple graphical way of displaying the status of the storage unit.
- Start button - starts the whole system
- Stop button - halts the program
- Conveyor belt start button - sends a command to the robot to move to the home position
- Inverter frequency buttons — three buttons to set inverter frequency to: 15 Hz, 20 Hz and 25 Hz.

5.8.1 Parameter calibration

An important feature of the GUI in this project is the ability to calibrate the HSV parameters, as well as erosion and dilation iteration number. This calibration allows to set and fine-tune the parameters for the image processing pipeline. The window for parameter calibration is shown in the figure 5.10, with the correct parameters set for masking.

Using the sliders, it is possible to adjust the lower and higher bounds for the HSV masking, as well as the erosion and dilation iteration count. To show how mask that needing adjustment looks like see figure 5.11. That mask needs adjusting because of cavities, which could cause instabilities and incorrect part detection in the vision algorithm. A proper mask needs to have corners as sharp as possible and have no holes or gaps.

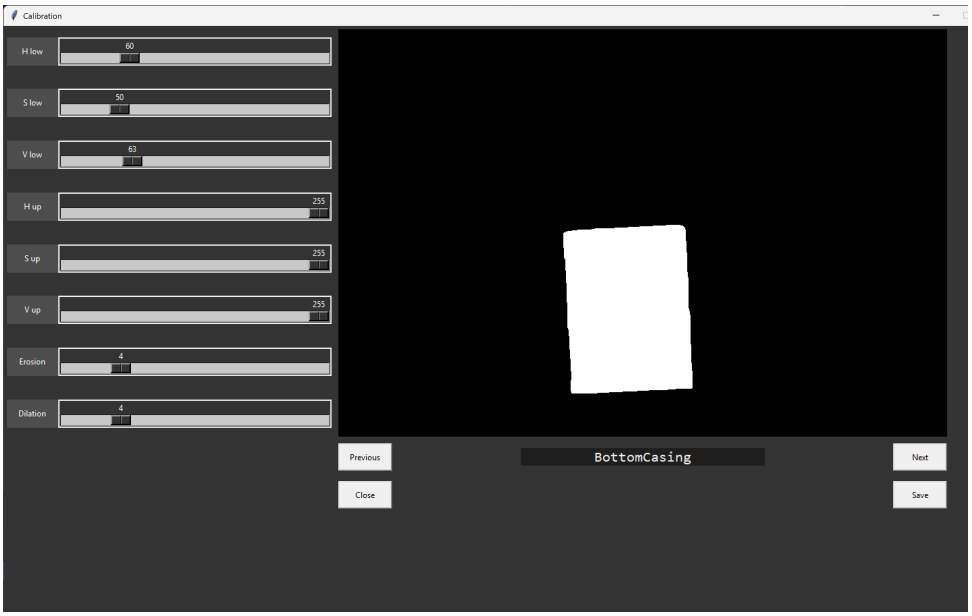


Figure 5.10: Calibration window with a correctly set mask

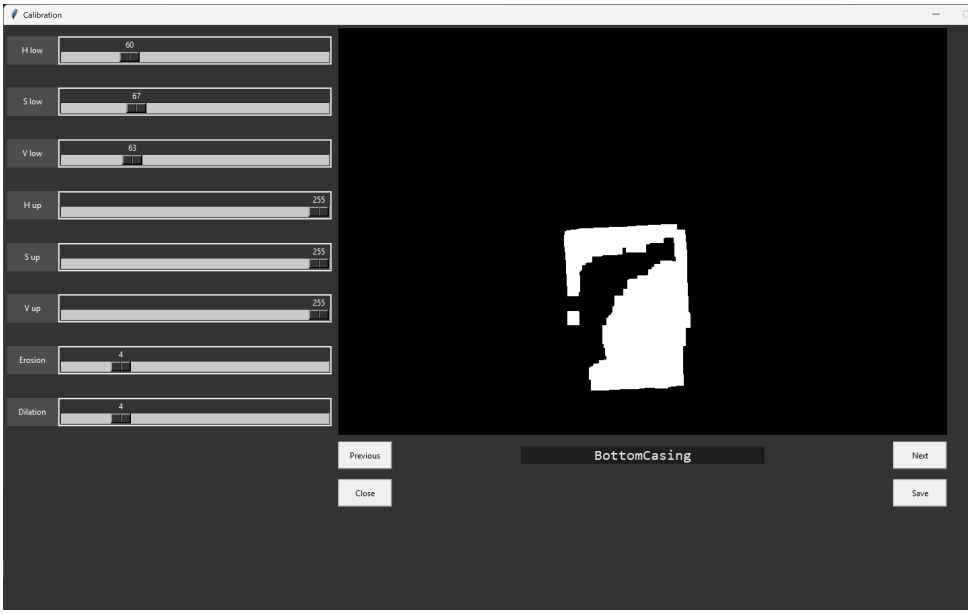


Figure 5.11: Calibration window with an incorrectly set mask

6

Control systems: robot motion and trajectory planning

This chapter overviews the motion and speed control of the robotic arm. Efficient control of the robotic arm's movement is important to quickly and accurately catch parts moving on the conveyor belt within a limited workspace (see Figure 1.1). Using the position estimated by the vision system (see Section 5.7) the robotic arm must intercept the moving part while accounting for its rotation (see Figure 6.1). The gripper has two fingers with flat pads (see Figure 1.2), and because the parts are rectangular, the finger pads must be alligated as close to parallel to the sides of the part as possible to make sure that the grasp is secure. The information about the rotation of the part is provided by the vision system designed in chapter 5.

6.1 Methodology

The methodology for robot motion control in this project focused mainly on achieving accurate and successful grasps of parts moving on the conveyor belt, rather than on speed. Based on insights from studies such as [13], the following methodology was developed for detecting, tracking, and catching objects from the conveyor belt. It involves four main steps:

State	Function	Linear axes	Rotational axes
1. Standby	The robot hovers above the conveyor and monitors for incoming parts.	None	None
2. Pursuit	Tracks the part using visual feedback and follows it along the conveyor.	X, Y	None
3. Descend	Once aligned, the robot descends to grasp the part from the conveyor.	Z	R_z
4. Placement	Lifts and moves the part to its designated slot in the storage unit.	X, Y, Z	R_z, R_y, R_z

Table 6.1: Robot states with corresponding functions as well as active linear and rotational axes.

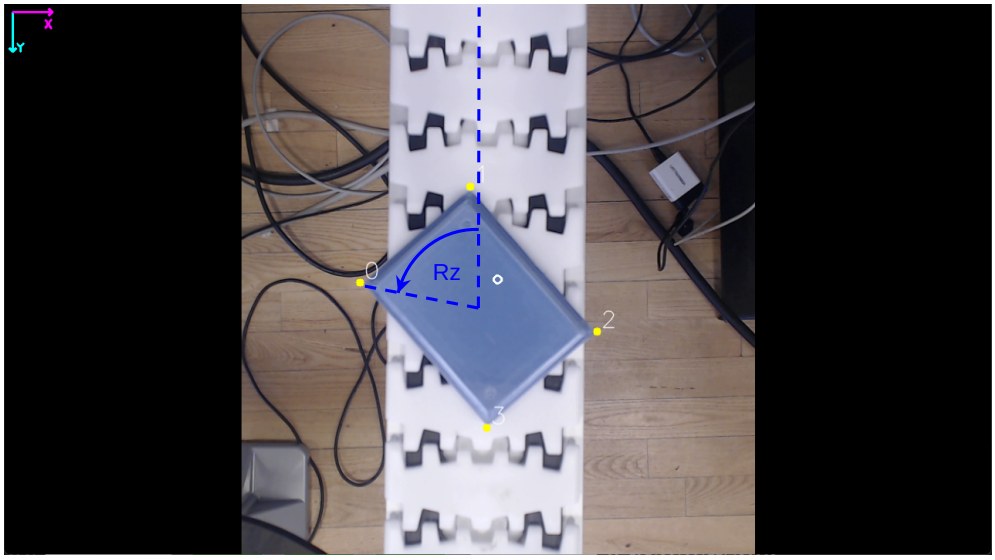


Figure 6.1: Example image of a rotated part viewed by the camera

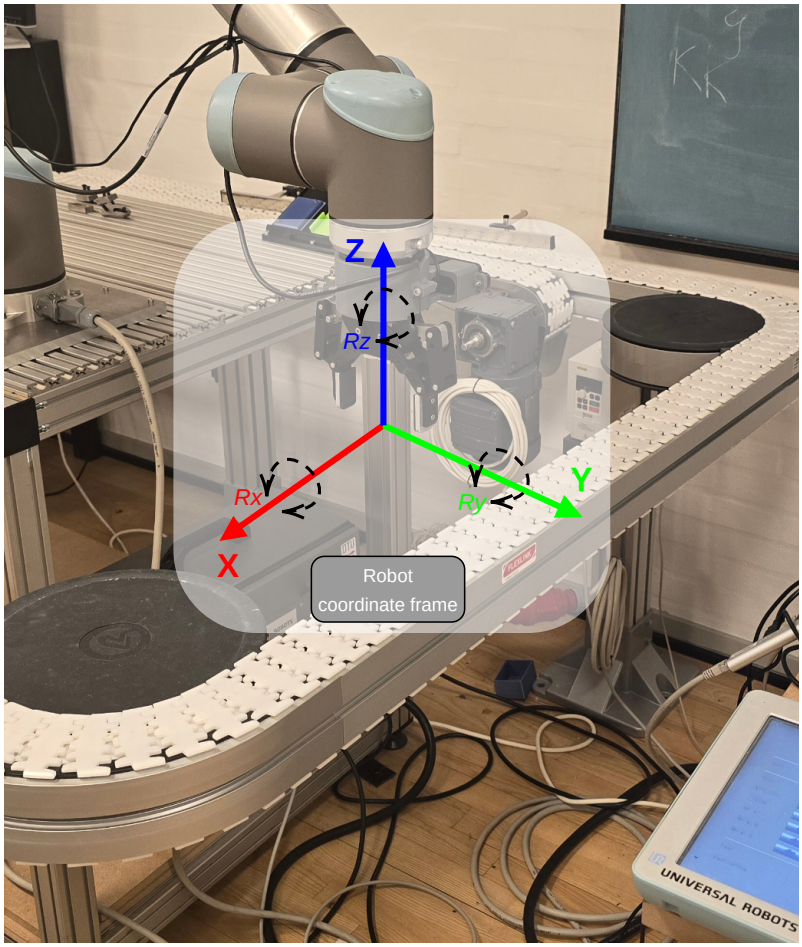


Figure 6.2: Overview of the axes of the robotic arm

6.2 Standby

In standby mode, the robot hovers stationary above the conveyor belt. The goal in this mode is to detect a part on the conveyor and calculate its angle of rotation (see Section 5.6) as well as the initial speed of the moving part. Estimating the initial speed is crucial for the subsequent algorithms to work correctly; its usage will be further discussed in section: 6.3.

6.2.1 Initial calculation of the part's velocity

The average velocity $x_{\text{speed,avg}}$ of the moving part is found by capturing several frames with a camera and figuring out the part's position in each frame. This velocity is measured along the robot's X-axis (see Figure 6.3). The average velocity measured by this method is negative, because the conveyor belt moves in the opposite direction to the robot's X-axis.

Only the third and third-from the last positions are used for the average velocity calculations. In this way potentially noisy measurements are avoided, which often occur at the beginning or end of the sequence. This usually happens, when the object is just entering or leaving the camera's field of view. The measured position can be then unstable due to noise or because the part is not yet fully visible.

To check the visibility of the part, the pixel area of the object is calculated and compared

to a fixed threshold. If this area is smaller than this threshold, the object is considered not fully visible, and the corresponding reading is discarded. However, this method does not entirely prevent noisy measurements. Therefore, some of the measurements from the start and the end of the sequence are still discarded.

This method is relatively simple and could be improved, but it proved to be reliable and good enough for this project. An alternative method would be to check the aspect ratio of the detected part and comparing it to a predefined reference value - big deviations from a predefined reference aspect ratio would indicate that the part is not fully visible. This approach also handles changes in the camera height better than the area method used in this project. However, the camera is always at the same height, so this change is unnecessary.

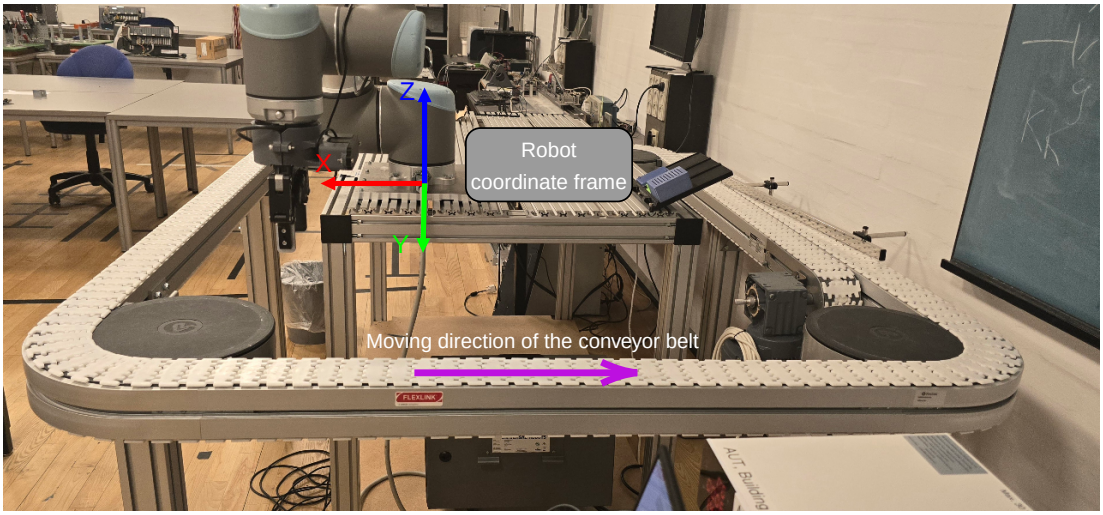


Figure 6.3: Robot's coordinate frame and the movement direction of the conveyor belt

The method for calculating average velocity for this project:

$$\Delta x = x_2 - x_{n-2}$$

where n is the number of captured data points, and x_2, x_{n-2} are the third and third-from-last position values in the array $x = \{x_0, x_1, x_2, \ldots, x_{n-1}\}$.

$$\Delta t = t_2 - t_{n-2}$$

$$v_{\text{avg}} = \frac{\Delta x}{\Delta t}$$

The final calculated average velocity is then used in the pursuit mode 6.3 for calculating parameters such as max speed or for predicting part position. A similar method was presented in a study from Florida State University [32]. Where authors used a comparable approach to estimate the speed of a moving object on the conveyor.

To verify that average velocity computed by this method is correct, it was compared to the actual conveyor belt speed measured and calculated by other methods that didn't rely on vision.

6.2.2 Reference velocity of the conveyor belt

The method used for reference velocity estimation of the conveyor belt involved measuring the time it takes for an object to travel one meter using a manual stopwatch. This method depended relied on how accurately a person could start and stop the stopwatch. So, the reliability was improved by taking five measurements for each tested frequency **??**. Those measurements were later averaged and used to calculate reference velocity for each conveyor belt frequency 6.3. The inverter frequencies tested: 15 Hz, 20 Hz and 25 Hz corresponded to the slow, medium and fast conveyor belt speeds.

Frequency (Hz)	**Time for 1 m travel (s)**					**Avg. Time (s)**
15	19.2	19.2	19.2	19.2	19.2	19.2
20	13.8	13.8	13.8	13.8	13.8	13.8
25	11.0	11.0	11.0	10.9	10.9	10.9

Table 6.2: Measured times for a 1 m travel distance at different inverter frequencies, rounded to one decimal place

The average velocity v_{avg} was calculated using the formula:

$$v_{\text{avg}} = -\frac{1}{t_{\text{avg}}}$$

To enable direct comparison with the velocity estimated by the vision system, the measured velocities have negative sign.

Frequency (Hz)	**Avg. Time (s)**	**Avg. Velocity (m/s)**
15	19.2	-0.052
20	13.8	-0.072
25	10.9	-0.092

Table 6.3: Average velocities for each analysed inverter frequency

To ensure that the reference velocity is as close to the truth value as possible another method was used to verify this value. The inverter frequency relation could be derived from mathematical calculations using the parameters of the motor and gearbox of the conveyor belt. According to the motor and gearbox specifications, WA20 DR63M4[1], the motor's shaft spins at 34 RPM (rotations per minute) when the inverter is set to a frequency of 50 Hz. Using this data and the radius of the conveyor belt drive wheel, we can calculate the actual conveyor belt speed with the following equation:

$$V\ (\text{m/s}) = \left(\frac{34\,\text{rpm}}{60\,s}\right) \times \frac{f}{50\,\text{Hz}} \times (2\pi r) \tag{6.1}$$

where:

- $\frac{34\,\text{rpm}}{60\,s}$ - converts revolutions per minute (rpm) to revolutions per second (rps),
- f - the inverter frequency in Hz,

[1] https://download.sew-eurodrive.com/download/pdf/19464819.pdf

- r - the radius of the conveyor wheel in meters.

The equation 6.1 assume a linear correlation between frequency set on the inverter and rotation of the motor's shaft. To verify it, a test was conducted to measure the RPM of the shaft. The method used, involved recording a video of the motor's shaft and analysing it to count the shaft rotations. The results are presented in table:

Frequency (Hz)	Time (s)	Rotations	RPM	RPM expected	Velocity (m/s)
15	82,04	14	10,24	10,2	0,0525
20	79,98	19	14,25	13,6	0,0731
25	86,60	26	18,01	17	0,0924

Table 6.4: Test of shaft's RPM

The table 6.4 confirmed that the correlation between frequency and motor's shaft RPM is linear.

Knowing that radius of the conveyor belt drive wheel is 49 mm, the calculated linear velocity of the conveyor belt at each inverter frequency is shown in table:6.5.

Frequency (Hz)	Calculated Velocity (m/s)
15	0.0525
20	0.0731
25	0.0924

Table 6.5: Calculated conveyor belt velocities using motor specifications

The velocity values from table 6.5 confirm that the velocities measured with the stopwatch method are close to the true values. Therefore, the velocity values from the table 6.3 were used as a reference for the measured velocities.

To verify the velocity measurements recorded from the vision system, 20 measurements per frequency and per part were recorded. These results are presented in the graphs below 6.4.

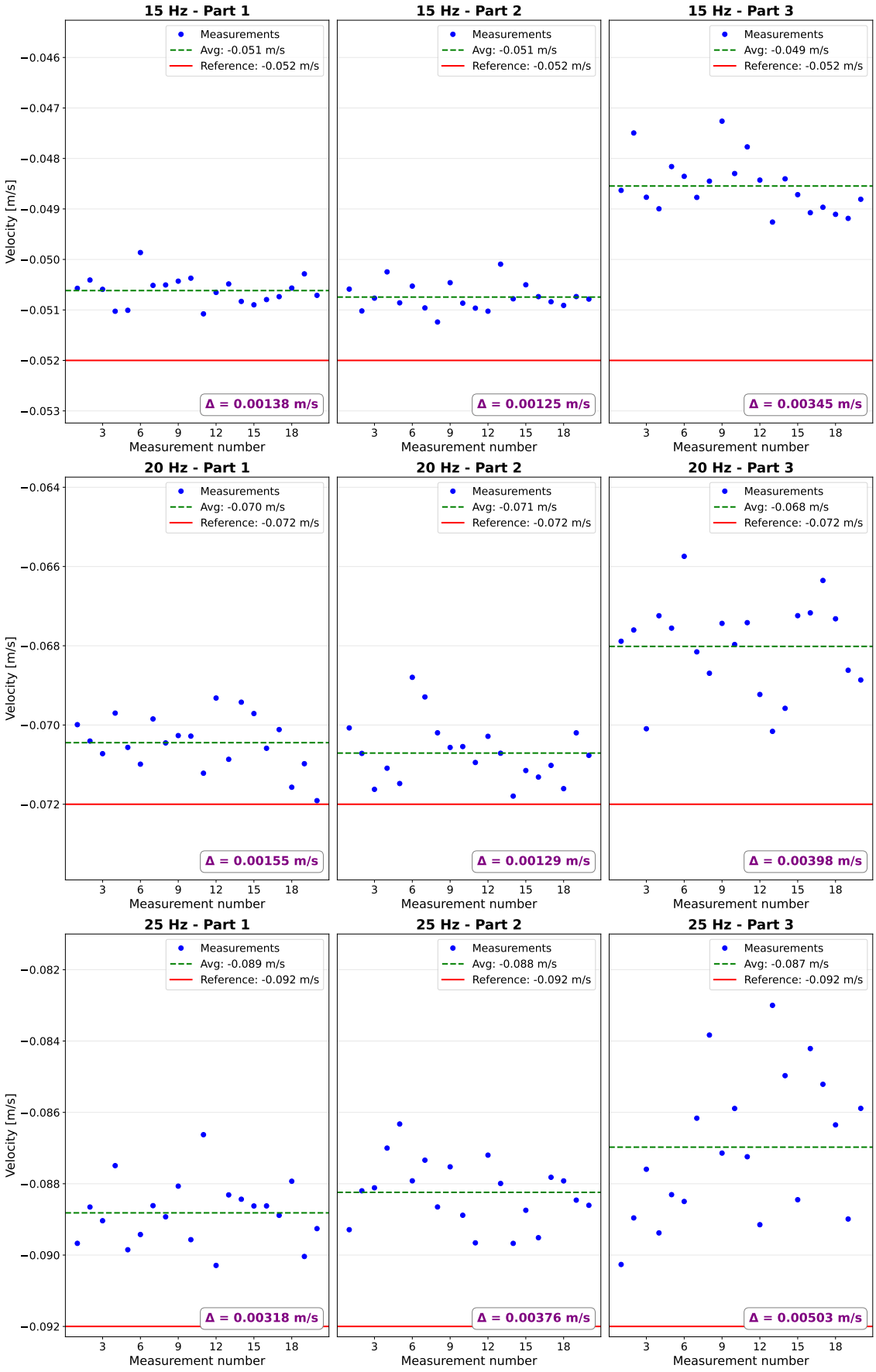


Figure 6.4: Velocities measured using the positions provided by the camera

To analyse the fidelity of the results, the mean measured velocity, standard deviation (STD), relative standard deviation (RSD), and bias relative to the reference value were calculated. Those values help with assessing any trends or systematic deviations can be found in the data gathered by the vision system.

Freq (Hz)	Part	Average velocity	Std	RSD (%)	Bias	Bias (%)
15	1	-0.05062	0.000288	0.57	-0.00148	-2.85
15	2	-0.05075	0.000276	0.54	-0.00135	-2.60
15	3	-0.04855	0.000550	1.13	-0.00355	-6.82
20	1	-0.07045	0.000689	0.98	-0.00205	-2.83
20	2	-0.07071	0.000764	1.08	-0.00179	-2.47
20	3	-0.06802	0.001166	1.71	-0.00448	-6.18
25	1	-0.08882	0.000892	1.00	-0.00248	-2.72
25	2	-0.08824	0.000914	1.04	-0.00306	-3.35
25	3	-0.08697	0.002051	2.36	-0.00433	-4.74

Table 6.6: Analysis of average velocity measurements

From Table 6.6, several trends can be observed.

Firstly, the system clearly underestimates the speed of the moving part, and never overestimates. As the inverter frequency increases, both standard deviation and relative standard deviation increases. This shows that the precision of the measurements decreases with increasing speed of the conveyor. This effect is likely due to less available time for the vision system to take position measurements - the higher the speed of the moving part the less measurements can be taken, reducing the quality of the assessed speed of the moving part.

Moreover, it is clear from the plots in figure 6.4 and the data in Table 6.6 that the estimated velocity for the third part is distinctly lower than the others. This is caused by the shadow cast by the part, affecting the vision system's measurement. When the part enters the camera's field of view the shadow is interpreted as part of the component due to using masking as an object detection algorithm, see Section 5.5. During the movement of the part the shadow changes size, which lowers the average speed calculated.

Additionally, the absolute bias for all the parts and frequencies remains below 0.005 m/s, which is acceptable for this applications. The robot always catches the part within 10 seconds, so this error would cause a maximum deviation of around 5 cm over that time.

Overall, the results are acceptable and provide a reliable initial velocity estimate.

6.3 Pursuit

The goal in the pursuit mode is to follow the part until the robot's gripper is directly above it to later grasp the part during the descend mode 6.4. During the pursuit mode, the robot tracks the part detected by the vision system using either real-time feedback from the camera or a predicted position.

6.3.1 Robust coordinates

To track a moving part on the conveyor belt, the robot is controlled using velocity vectors that are send continuously to the robot. Their calculation relies on the part's coordinates

provided by the vision system. To maintain stable control over the robot, these coordinates must be delivered in real time consistently and without delays.

The vision system may occasionally fail to deliver coordinates of the tracked part due to occlusions, motion blur or other image problems. To keep continuous tracking of the object, when the algorithm detects invalid coordinates from the vision system, it switches to using predicted coordinates. These are calculated from the last valid position its timestamp, as shown in equations (6.2) and (6.3).

However, a direct switch could potentially cause a noticeable jump in position used for the calculation of the velocity vectors for the robot, since predicted and measured positions may slightly differ. To prevent this, actual and predicted positions are smoothly blended using the alpha blending factor as illustrated by the equations bellow:

$$x_{\text{pred}} = (x_{\text{last}} + v_{\text{init}} \cdot (t_{\text{now}} - t_{\text{last}})) - (x_{\text{cam}} - x_{\text{robot}}) \tag{6.2}$$

$$y_{\text{pred}} = y_{\text{last}} + (y_{\text{cam}} - y_{\text{robot}}) \tag{6.3}$$

$$x_{\text{meas}} = x_{\text{cam}} \quad \text{(measured x-coordinate from the camera)} \tag{6.4}$$

$$y_{\text{meas}} = y_{\text{cam}} \quad \text{(measured y-coordinate from the camera)} \tag{6.5}$$

$$x_{\text{final}} = \alpha \cdot x_{\text{pred}} + (1 - \alpha) \cdot x_{\text{meas}} \tag{6.6}$$

$$y_{\text{final}} = \alpha \cdot y_{\text{pred}} + (1 - \alpha) \cdot y_{\text{meas}} \tag{6.7}$$

$$\alpha = \begin{cases} \alpha - \Delta\alpha, & \text{if valid detection from the camera} \\ \alpha + \Delta\alpha, & \text{otherwise} \end{cases} \tag{6.8}$$

$x_{\text{pred}}, y_{\text{pred}}$	predicted positions.
$x_{\text{meas}}, y_{\text{meas}}$	measured positions from the vision system.
$x_{\text{final}}, y_{\text{final}}$	positions used for robot control.
$x_{\text{last}}, y_{\text{last}}$	last valid positions.
v_{init}	estimated part velocity.
$t_{\text{now}}, t_{\text{last}}$	current and previous timestamps.
α	blending factor.

The alpha blending factors works as a smoothing parameter that gradually adjusts the weights between the predicted and measured positions, whenever the source of the position switches. This approach reduces sudden changes and ensures that the positions from both sources smoothly combined. This method is similar to the complementary filter used during the sensor fusion [33]. Complementary filters merge different data sources by blending them over time, which prevents rapid transitions from occurring and improves the overall stability.

To validate the performance of this method the positions and speeds plots can be analysed to check if any big speed jumps occur - this would indicate that the method is not blending the predicted and measured positions smoothly.

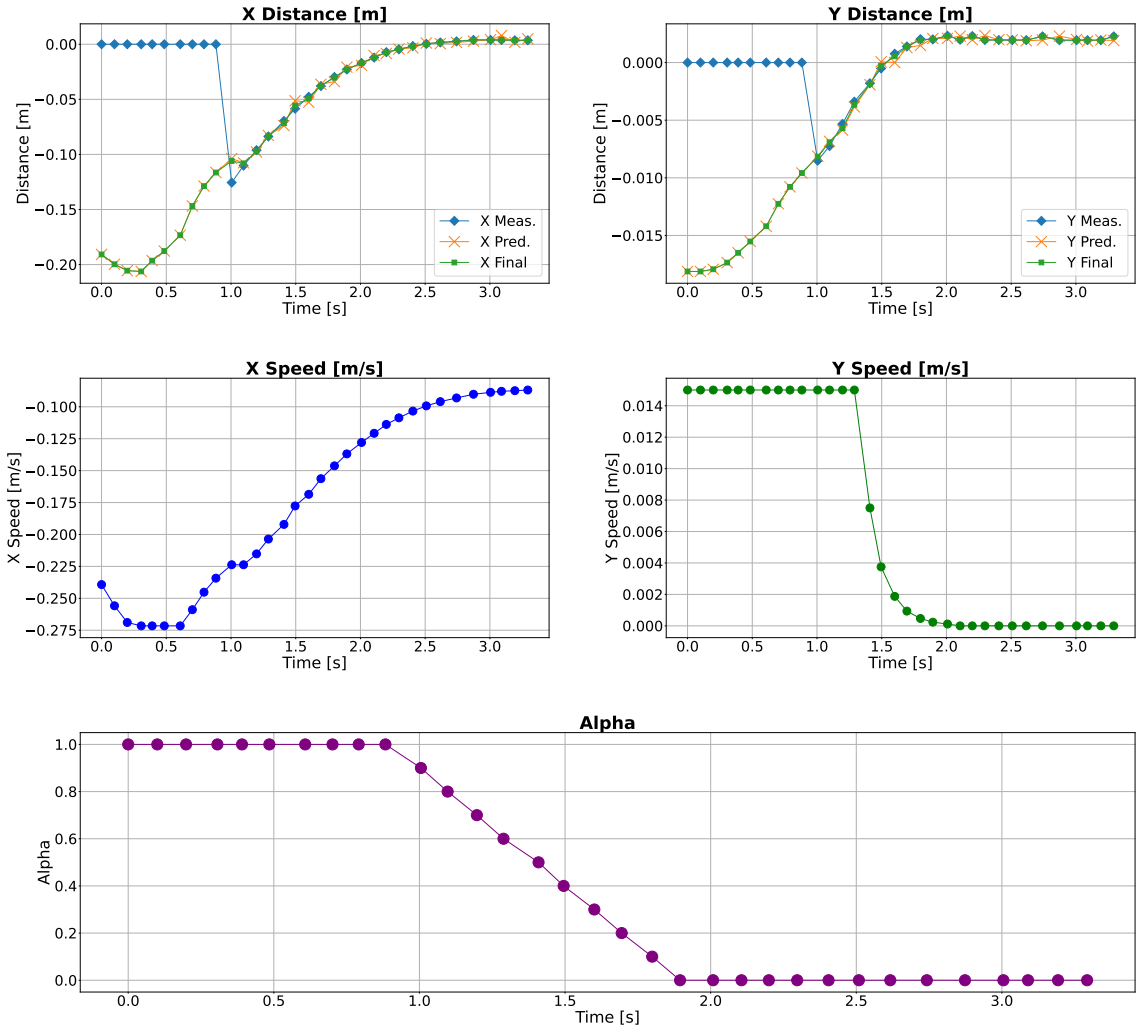


Figure 6.5: Comparison of x and y speeds as well as alpha variable

Figure 6.5 illustrates an example run of the system and how the X-distance (see figure 6.6), Y-distance (see figure 6.7), v_x, Y-speed, and the alpha value changes over time. At the beginning, the measurements from the vision system (X meas. and Y meas.) are zero, because the part is not in the camera's FOV (field of view). During this period the system must rely on the predicted position of the part. When the part finally comes into the camera's FOV at around 1 second mark the measured values begin to blend with the predicted values for both axes. Even thought, there is small deviation between measured predicted X values there is no sudden jump in the v_x. Instead, it decreases gradually, which indicates that the alpha blending is working correctly. The same is observed for the Y-speed. Moreover, the values for the predicted and measured positions align very closely, confirming that the accuracy of the estimated position.

6.3.2 Speed control along X and Y axes

The coordinates $(x_{\text{final}}, y_{\text{final}})$ described in the previous method, are continuously used during the pursuit mode. During this mode the robot moves along the negative X-axis direction using PID controller, while making small adjustments the Y position to account

for any shifts along the Y-axis. The Z-axis distance remains constant, ensuring that the part is fully visible during the movements of the robot.

The goal is to minimize the error (distance along the X-axis from the gripper to the part, as shown in figure 6.6).

The initial speed calculated by the method described in the section 6.2.1 is used in this mode to calculate the maximal v_x for the robotic arm 6.9.

$$v_{\text{x},max} = v_{\text{x, initial}} + \frac{(x_{\text{last}} + v_{\text{x, initial}} \cdot (t_{\text{now}} - t_{\text{last}}))}{T_{catch}} \tag{6.9}$$

This equation estimates the speed required to catch up to the part within T_{catch} seconds and sets the maximum allowable velocity for the robot. This step is important because at the start of the pursuit mode the robot is moving at high speeds, and setting the maximum speed proportional to the actual speed of the part reduces the risk of overshoot.

$v_{\text{x, max}}$ maximum allowed velocity in the X direction.
$v_{\text{x, initial}}$ initial estimated velocity of the part.
x_{last} last known X position of the part.
t_{now} current timestamp.
t_{last} last known timestamp of the part, when it was visible by the camera.

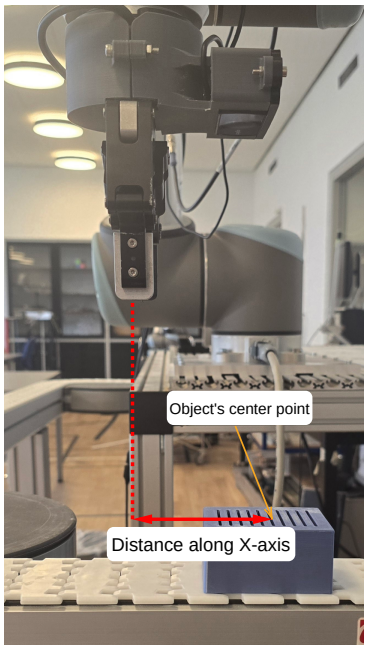


Figure 6.6: Distance to the object along X-axis

Speed control along X-axis

To control the speed of the robotic arm along the X-axis, a PID controller is used. The equation 6.10 explains how v_x is calculated. The information about control theory and PID's controllers was taken from [34].

$$v_x = K_p \cdot e_x + K_i \cdot \int e_x dt + K_d \cdot \frac{de_x}{dt} \tag{6.10}$$

v_x	velocity along X-axis.
e_x	distance to the part in meters.
$K_p \cdot e_x$	proportional term.
$K_i \cdot \int e_x \, dt$	integral term - calculates the accumulated error over time.
$K_d \cdot \frac{de_x}{dt}$	derivative term - estimates the rate of change of the error over time.

The PID controller used in this projects uses positional error e_x (measured in meters) to estimate the output velocity v_x (measured in meters per second). Therefore it's highly dependant on the integral term, since the proportional term becomes very small as the robot approaches the target. Without the integral term, the controller would output velocity close to zero, which would cause erratic and unpredictable behaviour of the robot.

The tuning of the PID controller had to be performed in such a way that the integral term, that accumulates the error over time, maintains enough velocity so the robot doesn't stall, when the proportional term is very low.

After suitable proportional and integral terms were found, through trail-and-error testing, the derivative term was tuned. The derivative term helps the robot reach the target faster. The derivative term is large when the error changes rapidly, and smaller when the rate of change is low. This effectively increases the robot's speed at the beginning when the distance is large. As the robot is near the target and the rate of change slows down the derivative term decreases and contributes less to the velocity of the robot. However, the derivative term is often noisy and this could lead to unstable velocity control. To solve this issue low-pass filter is often implemented in PID controllers to mitigate this problem.

The derivative term is calculated and passed through a first-order low-pass filter with time constant τ_d to reduce the high-frequency noise.

$$\frac{de_x}{dt} = \frac{e_x - e_{\text{x,prev}}}{dt} \tag{6.11}$$

$$\alpha = \frac{\tau_d}{\tau_d + dt} \tag{6.12}$$

$$d = \alpha \, d_{\text{prev}} + (1 - \alpha) \, \frac{e_x - e_{\text{x,prev}}}{dt} \tag{6.13}$$

$$e_{\text{prev}} = e_x \tag{6.14}$$

$$d_{\text{prev}} = d \tag{6.15}$$

$\frac{de_x}{dt}$	unfiltered derivative term.
e_x	distance to the part in meters.
α	filter coefficient, controls derivative smoothing.
τ_d	low-pass filter time constant.
dt	control loop time step.
d_{prev}	derivative value from the previous iteration.
e_{prev}	error value from the previous iteration.

Speed control along the Y-axis

Since the position of the part on the conveyor belt is constant during its movement, the speed control along the Y-axis can be easily controlled without using a PID controller. The position of the part along the Y-axis is constant, because the X-axis is parallel to the movement direction of the conveyor belt (see Figure 6.3). Therefore to control the Y-speed a simple threshold-based control is used.

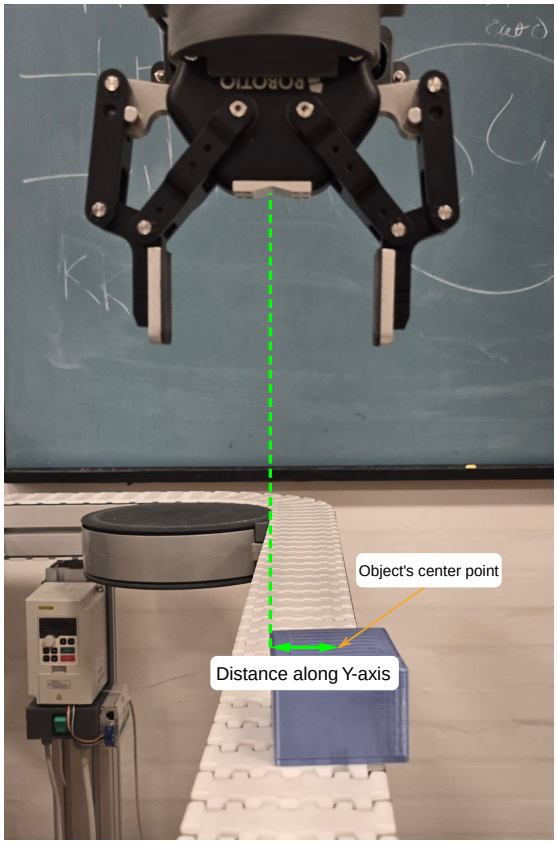


Figure 6.7: Distance to the object along Y-axis

$$v_{\text{y}} = \begin{cases} v_{\text{y, max}} \cdot \text{dir}_y, & \text{if } |y_{\text{distance}}| > y_{\text{threshold}} \wedge v_{\text{y, goal}} = \text{False}, \\ 0.5 \cdot v_{\text{y}}, & \text{otherwise, with } v_{\text{y, goal}} = \text{True}, \\ 0, & \text{if } |v_{\text{y}}| < 1 \cdot 10^{-4}. \end{cases}$$

v_{y}	Current speed of the part along the Y-axis.
$v_{\text{y, max}}$	Maximum allowed speed along the Y-axis.
$v_{\text{y, goal}}$	Boolean flag indicating if the target goal is achieved.
y_{distance}	Distance to the part along the Y-axis.
$y_{\text{threshold}}$	Threshold distance to stop the speed adjustment.
dir_y	Direction of the required correction along the Y-axis (either +1 or -1).

6.4 Descend

In the descend mode the robot moves at the v_x estimated by the PID controller descending along the Z-axis 6.8 to grasp the moving part, while rotating the end-effector to align the gripper with the part's orientation. During this movement the camera doesn't give a feedback to the system and the robot moves only according to the estimates of the v_x and part rotation.

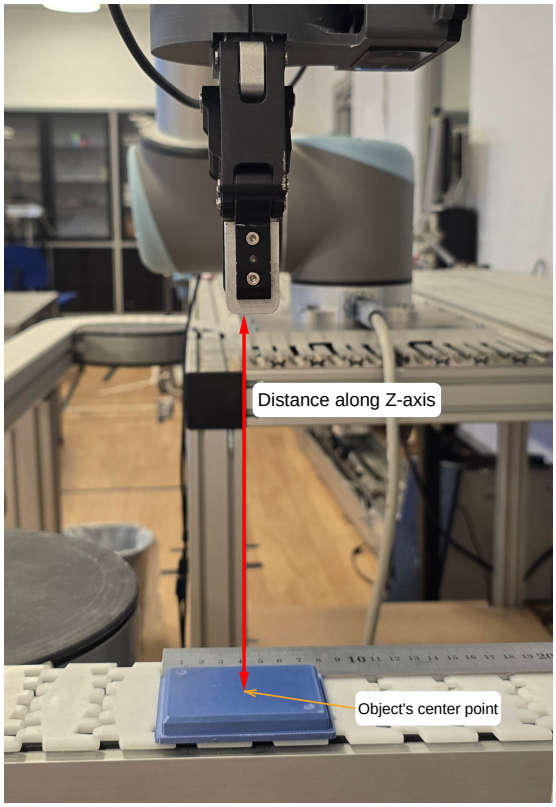


Figure 6.8: Distance along the Z-axis to the part

The distance to the part is calculated using this equation:

$$z_{\text{distance}} = \max(z_{\text{pose}} - h_{\text{descend}}, 0) \tag{6.16}$$

z_{distance}	Distance to the part.
z_{pose}	Z-axis position of the robotic arm (see Section 4.1.2).
h_{descend}	Target Z-position, set in the program.
max	Function that returns the bigger value of the two.

The z_{distance} is needed to calculate the velocity of the robotic arm during the descend. The equation for calculating v_z is as follows:

$$v_{\mathsf{z}} = \begin{cases} \max\left(v_{\mathsf{z}} - v_{\mathsf{z,\,increment}},\quad v_{z,\mathsf{max}}\right), & \text{if: } z_{\mathsf{distance}} > z_{\mathsf{half}} \\ \max\left(v_{\mathsf{z}} \cdot \sqrt{\min\left(\max\left(\frac{z_{\mathsf{distance}}}{z_{\mathsf{half}}}, 0\right), 1\right)},\quad v_{z,\mathsf{max}}\right), & \text{if: } 0.001 < z_{\mathsf{distance}} \leq z_{\mathsf{half}} \end{cases} \tag{6.17}$$

v_{z} Velocity of the robotic arm along the Z-axis (begins at 0 m/s).
$v_{\mathsf{z,\,increment}}$ Z-axis position of the robotic arm (see Section 4.1.2).
$v_{\mathsf{z,\,max}}$ Maximal allowed v_z.
z_{half} Half of the distance between the part and the robotic arm (calculated once before the des

The descend begins by using the $v_{\mathsf{z,\,increment}}$ to gradually increase the speed of the robotic arm, until the robot hits the half point of the descend z_{half}. Afterwards, the speed of

$$dt_\theta = t_{\theta,\,\mathsf{current}} - t_{\theta,\,\mathsf{last}} \tag{6.18}$$
$$t_{\theta,\,\mathsf{last}} = t_{\theta,\,\mathsf{current}} \tag{6.19}$$
$$r_z = r_z - v_{r_z} \cdot dt \tag{6.20}$$

$$\theta_{\mathsf{accel}} = 0.1 \cdot \theta_{\mathsf{target}} \tag{6.21}$$
$$\theta_{\mathsf{decel}} = 0.9 \cdot \theta_{\mathsf{target}} \tag{6.22}$$

$$p_{\mathsf{accel}} = \min\left(\max\left(\frac{|r_z|}{\theta_{\mathsf{accel}}}, 0\right), 1\right) \tag{6.23}$$
$$p_{\mathsf{decel}} = \min\left(\max\left(\frac{|r_z - \theta_{\mathsf{decel}}|}{|\theta_{\mathsf{target}} - \theta_{\mathsf{decel}}|}, 0\right), 1\right) \tag{6.24}$$
$$s = -\mathsf{sign}(\theta_{\mathsf{target}}) \tag{6.25}$$

$$v_{r_z} = \begin{cases} \max\left(s \cdot v_{\mathsf{max}}\left(1 - e^{-4 \cdot p_{\mathsf{accel}}}\right),\ s \cdot v_{\mathsf{min}}\right), & |r_z| < |\theta_{\mathsf{accel}}| \\ v_{r_z} \cdot \left(1 - (1 - p_{\mathsf{decel}})^2\right), & |r_z| \geq |\theta_{\mathsf{decel}}| \wedge |v_{r_z}| > 1 \cdot 10^{-4} \\ 0, & |r_z| \geq |\theta_{\mathsf{decel}}| \wedge |v_{r_z}| \leq 1 \cdot 10^{-4} \\ v_{r_z}, & \text{otherwise} \end{cases} \tag{6.26}$$

6.4.1 Placement

During the placement phase of the robot control the part is moved from the conveyor belt to a designated slot in the part holder. The system slows the robot after the pursuit mode is finished and simultaneously ascends. The equations used to do it are presented in the equations below.

$$t = \mathsf{t}_{now} - \mathsf{t}_{start} \leq T_{\mathsf{ascend}} \tag{6.27}$$

$$T_{\text{ascend}} = 2s \tag{6.28}$$

$$v_x = \begin{cases} 0, & \text{if: } v_x \geq -10^{-4} \\ 0.98 \cdot v_x, & \text{otherwise} \end{cases} \tag{6.29}$$

$$v_z = \begin{cases} \min\left(v_z(t-\Delta t) + v_{z,\text{ increment}},\ z_{\max}\right), & t < \frac{T_{\text{ascend}}}{2} \\ \max\left(v_z(t-\Delta t) - v_{z,\text{ increment}},\ 0\right) & t \geq \frac{T_{\text{ascend}}}{2} \\ or \quad \text{if: } v_z(t) \leq 1 \cdot 10^{-4},\ \text{then } 0 & \end{cases} \tag{6.30}$$

After successfully completing the ascend, the robot goes to the part storage unit (see figure 6.9) where it places the part in its designated spot.

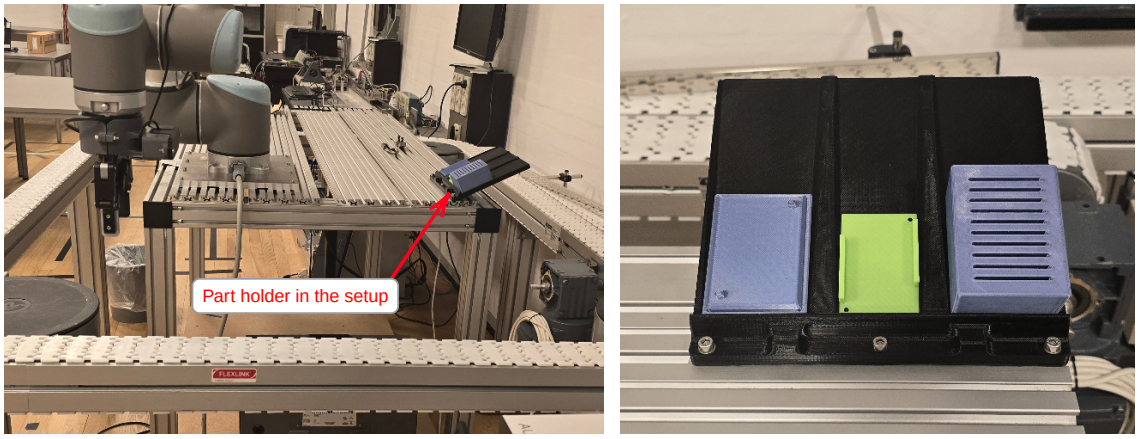


Figure 6.9: Part holder

7

System test

This chapter presents the outcome of the system-level tests performed to evaluate the work done in this project. It's also meant to verify that all the objectives defined in Chapter 2 1.2 have been met.

7.1 Performance tests

A well-designed robotic system for pick-and-placement applications should have a success rate close to 100%. Meaning that the robot picks almost all the parts from a conveyor belt and places them in their designated locations.

To test the reliability of the designed robotic system, a performance test was conducted assessing the pickup rate based on the number of successful pickups for each part at various conveyor belt speeds.

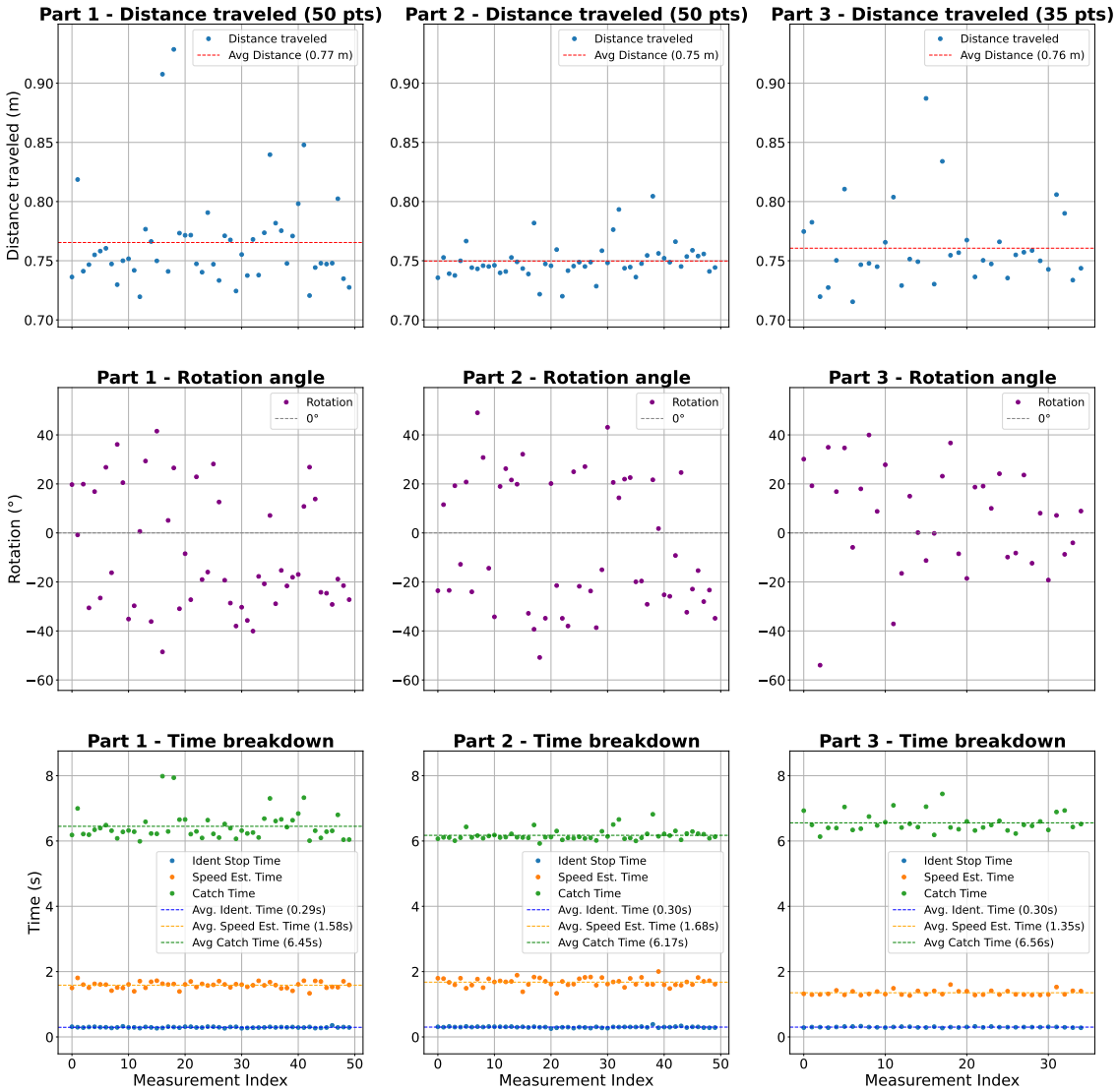


Figure 7.1: Performance tests for inverter frequency of 25 Hz

8

Conclusion

The conclusion should list the positive results of the thesis and set the results in the perspective of the objectives.

(Future work - things not finished or not working - should preferably be somewhere else, e.g. in the previous chapter).

Bibliography

[1] *Industrial Process Improvement by Automation and Robotics*. 2023. URL: https://www.mdpi.com/2075-1702/11/11/1011.

[2] *Robot Guidance Using Machine Vision Techniques in Industrial Environments: A Comparative Review*. 2016. URL: https://www.mdpi.com/1424-8220/16/3/335.

[3] *Challenges and Benefits of Industry 4.0: an overview*. 2018. URL: https://www.researchgate.net/publication/330668156_Challenges_and_Benefits_of_Industry_40_an_overview.

[4] *Adaptable Distributed Vision System for Robot Manipulation Tasks*. 2025. URL: https://www.scitepress.org/Papers/2025/131156/131156.pdf.

[5] *A Robotic Automatic Assembly System Based on Vision*. 2020. URL: https://www.mdpi.com/2076-3417/10/3/1157.

[6] *Sensor-Guided Assembly of Segmented Structures with Industrial Robots*. 2021. URL: https://www.mdpi.com/2076-3417/11/6/2669?.

[7] *An Innovative Vision-Guided Feeding System for Robotic Picking of Different-Shaped Components*. 2024. URL: https://www.mdpi.com/2227-7080/12/9/153.

[8] Puchuan Song. *Object Detection based on HSV in OpenCV*. 2025. URL: https://www.ewadirect.com/proceedings/ace/article/view/19740.

[9] *Object Tracking using Adaptive Template Matching*. 2015. URL: https://www.researchgate.net/publication/277648222_Object_Tracking_using_Adaptive_Template_Matching.

[10] *You Only Look Once: Unified, Real-Time Object Detection*. 2016. URL: https://arxiv.org/abs/1506.02640.

[11] *Computer Vision-Based Robotic System Framework for the Real-Time Identification and Grasping of Oysters*. 2025. URL: https://www.mdpi.com/2076-3417/15/7/3971.

[12] *Visual control of robot manupulators - a review*. URL: https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=c02adc49d7027dee3f04b8bc159f8cac9530f705.

[13] *Visual Servoing for Object Manipulation: A Case Study in Slaughterhouse*. 2016. URL: https://backend.orbit.dtu.dk/ws/portalfiles/portal/130245523/PID4391789_proofed.pdf.

[14] *Robot Visual Servo through Trajectory Estimation of a Moving Object Using Kalman Filter*. 2009. URL: https://www.researchgate.net/publication/220776549_Robot_Visual_Servo_through_Trajectory_Estimation_of_a_Moving_Object_Using_Kalman_Filter/citations.

[15] *Target Tracking for Visual Servoing Systems Based on an Adaptive Kalman Filter. International Journal of Advanced Robotic Systems*. 2017. URL: https://journals.sagepub.com/doi/10.5772/52035?.

[16] *Robotic eye-in-hand calibration by calibrating optical axis and target pattern*. 1995. URL: https://link.springer.com/article/10.1007/BF01258384.

[17] *Robotic cell assembly to accelerate battery research*. 2022. URL: https://pubs.rsc.org/en/content/articlelanding/2022/dd/d2dd00046f.
[18] *FieldNet: Efficient real-time shadow removal for enhanced vision in field robotics*. 2025. URL: https://www.sciencedirect.com/science/article/pii/S0957417425010644.
[19] Ramir Sultanov et al. *A Review on Collaborative Robots in Industrial and Service Sectors*. 2022. DOI: 10.1109/SIBCON56144.2022.10003014.
[20] *34759 Perception for Autonomous Systems*. 2024. URL: https://kurser.dtu.dk/course/34759.
[21] *A Comparative Review of Hand-Eye Calibration Techniques for Vision Guided Robots*. 2021. URL: https://ieeexplore.ieee.org/abstract/document/9512030.
[22] *Camera Calibration in High-Speed Robotic Assembly Operations*. 2024. URL: https://www.mdpi.com/2076-3417/14/19/8687#B15-applsci-14-08687.
[23] *Interpretation and Transformation of Intrinsic Camera Parameters Used in Photogrammetry and Computer Vision*. 2022. URL: https://www.mdpi.com/1424-8220/22/24/9602.
[24] *Extraction and analysis of microscopic traffic data in disordered heterogeneous traffic conditions*. 2019. URL: https://www.researchgate.net/publication/337558917_Extraction_and_analysis_of_microscopic_traffic_data_in_disordered_heterogeneous_traffic_conditions.
[25] Zhengyou Zhang. *A Flexible New Technique for Camera Calibration*. 1998. URL: https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/tr98-71.pdf.
[26] *Methods for Simultaneous Robot-World-Hand–Eye Calibration: A Comparative Study*. 2019. URL: https://www.mdpi.com/1424-8220/19/12/2837.
[27] *Calibration of wrist-mounted robotic sensors by solving homogeneous transform equations of the form AX=XB*. 2002. URL: https://ieeexplore.ieee.org/document/88014.
[28] *A New Technique for Fully Autonomous and Efficient 3D Robotics Hand/Eye Calibration*. 1989. URL: https://kmlee.gatech.edu/me6406/handeye.pdf.
[29] *Dex-Net 2.0: Deep Learning to Plan Robust Grasps with Synthetic Point Clouds and Analytic Grasp Metrics*. -. URL: https://www.roboticsproceedings.org/rss13/p58.pdf.
[30] *HSV Color-Space-Based Automated Object Localization for Robot Grasping without Prior Knowledge*. 2021. URL: https://www.mdpi.com/2076-3417/11/16/7593.
[31] OpenCV. *Camera Calibration and 3D Reconstruction*. 2025. URL: https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html?.
[32] *Conveyor visual tracking using robot vision*. 2006. URL: https://www.researchgate.net/publication/228670939_Conveyor_visual_tracking_using_robot_vision.
[33] *Sensor Fusion of GNSS and IMU Data for Robust Localization via Smoothed Error State Kalman Filter*. 2023. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC10099052/.
[34] Gene F. Franklin. *Feedback control of dynamic systems*. 2020. URL: https://mrce.in/ebooks/Feedback%20Control%20of%20Dynamic%20Systems%208th%20Ed.pdf.